UNIVERSITY OF MACEDONIA

SCHOOL OF INFORMATION SCIENCES

DEPARTMENT OF APPLIED INFORMATICS

BATTLING THE SKILLS GAP: IDENTIFICATION OF JAVA KNOWLEDGE UNITS THROUGH MACHINE LEARNING

Bachelor's Thesis

of

Nikolaos Papadopoulos

Thessaloniki, July 2024

BATTLING THE SKILLS GAP: IDENTIFICATION OF JAVA KNOWLEDGE UNITS THROUGH MACHINE LEARNING

Nikolaos Papadopoulos

Undergraduate Student of Applied Informatics at the University of Macedonia

Bachelor's Thesis

submitted for the partial fulfillment of its requirements

BSc in Applied Informatics, Computer Science & Technology

Supervisor
Alexander Chatzigeorgiou

Approved by the three-member examination committee on 09/07/2024

Alexander Chatzigeorgiou	Apostolos Ampatzoglou	Stylianos Xinogalos
.....	.....	.....

Nikolaos Papadopoulos

.....

Abstract

'Change is the only certainty in tech'. The maxim captures the rapid changes in the demand of skills in IT and software development in particular. The expertise that software professionals possess evolves throughout their career, reflecting trends in the dominant technology stacks. Machine Learning can be put to good use for identifying fine-grained skills based on the artifacts that software professionals develop. In this thesis we present a methodology that leverages classification techniques to label individual commits in software repositories and track both the technologies owned by each contributor and the extent of technology used at different points in time. To this end, we employ the notion of Knowledge Units, representing a cohesive set of key capabilities in a given programming language. The achieved classification accuracy enables the early and automated identification of skill gaps between employed technologies in an organization and the knowledge possessed by the corresponding workforce as well as a detailed mapping of knowledge distribution among the members of a development team. We complement our study with a demo tool that allows users to put our proposed methodology into practice. It provides an easy way to analyze repositories and extract the detected skills.

Keywords: programming skills, Java Knowledge Units, Machine Learning, Neural Networks, CodeBERT, skill identification

Acknowledgment

First and foremost, I'd like to extend my sincerest gratitude to my professor and supervisor, Dr. Alexander Chatzigeorgiou, for his continuous support and guidance throughout this project. The excitement and interest he showed in the subject as well as his willingness to help were motivating and I'm glad we had the opportunity to work together.

To my dear friend Stavroula Tsoni, a big thank you for her assistance with the research and experimentation on neural networks, and for being one of the most capable and hard-working people I had the pleasure of working with.

And heartfelt thanks to my parents, sister, and grandmother, for their unwavering support throughout my academic journey—and throughout the years—that allowed me to fully focus on my studies and achieve milestones that otherwise would not have been possible.

As this thesis reaches its conclusion, so does a chapter of my life—my undergraduate studies. Looking back, the most important and valuable thing that I obtained from this journey was the people I met. I'd like to thank all the friends, colleagues, and teachers who have been a part of this and have contributed—directly or indirectly—to making this a remarkable experience.

Contents

List of Figures

List of Tables

1 Introduction

In the ever-evolving domain of information technology (IT), the dynamic landscape consistently reshapes the demand for specialized skills, especially within software development. As technologies progress, software professionals adapt, aligning their expertise with the latest trends. This ongoing evolution underscores the importance of a flexible skill set for the individual. While an adaptive skill set benefits the professional, identifying and quantifying these skills becomes of paramount importance for employers, especially in an era where labor shortages in the ICT sector are persistent [3], [4], [5]. Such identification aids in decision-making, resource allocation, and workforce development:

- It is essential for managers to know their team's experience and skills, as finding developers with the necessary skills for a project is a critical aspect of strategic decision-making [6].
- Knowing the skills of employees is crucial for selecting the right person for the right job, which is a core part of resource allocation in software development projects [7]. Effective management and planning can drastically influence a project's flow, and are fundamental for the success of these projects.
- Skill identification is the first step towards addressing skill gaps. Employers are recognizing existing skill gaps or anticipating them in the near future. In a survey, 44% of respondents said their organizations would face skill gaps within the next five years, while another 43% reported already existing skill gaps [8]. The timely identification of skill shortages can then drive effective reskilling programs.

1.1 Challenges in Skill Assessment

Traditional methods for assessing a software professional's expertise, such as self-reporting or periodic reviews, have their merits but often fall short of capturing the multifaceted and evolving nature of a developer's skill set. For example, HR interviews

can only capture specific snapshots in the career of a professional and can hardly focus on low-level competencies such as the knowledge of a specific library or API. With the broad array of technologies and languages that a developer might engage with over their career, there's an evident need for a more refined, objective, and scalable mechanism for skill assessment.

1.2 Role of Machine Learning in Skill Assessment

Machine Learning (ML) offers a nuanced approach to this challenge. In the context of software development, the artifacts (i.e., source code) produced by developers can serve as significant indicators of their competencies. This thesis proposes a methodology leveraging ML to label individual software commits, providing insights into the specific technologies a contributor is adept in and tracing the trajectory of their professional development as it is mirrored in their source code contributions.

Central to our approach is the introduction of the term "*Knowledge Unit*" (KU). As stated in the study by Ahasanuzzaman et al. [2], "*We define a KU as a cohesive set of key capabilities that are offered by one or more building blocks of a given programming language*". The selection of KUs, aids in refining the analysis by marking out distinct areas of expertise within the vast domain of software development. Binary classifiers and neural networks (NNs) are then used to detect KUs in files affected by changes during a commit. The main contributions of the proposed approach can be summarized as follows:

- An extensive training dataset of code snippets including the KUs of interest has been created with the use of ChatGPT and then refined through keyword search and preprocessing to emphasize the code's structural and syntactic features.
- Exhaustive experimentation was carried out with various binary classifiers and neural networks along with k-fold cross validation.
- KU detection was performed employing a sliding window approach to enhance the likelihood of suitably aligning a KU within a window spanning various lines of code.

- The evaluation of KU identification with the sliding window technique was performed using thousands of code files from actual GitHub repositories, employing regular expressions for labeling true KU occurrences.

- A demo front-end web app was developed to allow easier interaction with the KU detection tool.

The potential applications of our methodology go beyond individual skill assessment. For businesses and organizations, this deep dive into skill analytics can reshape strategies. Whether it is identifying training needs, making hiring decisions to fill expertise voids, or reallocating human resources to match project requirements, the data-driven insights pave the way for smarter, more informed choices. In a domain where the only constant is change, having a clear map of the skills terrain becomes an invaluable asset.

1.3 Rationale for our Approach

While it might make sense to think that the deterministic nature of static code analysis ensures results so precise that ML methodologies may find it challenging to compete, this perspective merits further examination. Static code analysis undoubtedly offers a high degree of confidence in the identification of code artifacts, particularly for rudimentary or intermediate functionalities, such as the utilization of operators or the construction of loops. However, its efficacy diminishes when confronted with intricate realms, like the services inherent in Java EE. Crafting exhaustive rules and patterns to cover these complexities demands the intervention of domain experts, a process that is both labor-intensive and potentially lacking in capturing nuances. Furthermore, with the continuous evolution of coding practices, languages, frameworks and libraries, it becomes increasingly difficult to persistently update existing rules or instantiate new ones.

Machine Learning can provide a way to reduce the extensive effort required to build such systems, while maintaining an adequate level of accuracy in the detection of KUs. ML can capture intricate correlations and patterns present in data which might

be challenging to codify into rules. Additionally, as new patterns become available, they could be integrated into the system by fine-tuning existing models, with relatively limited effort assuming that a sufficient dataset for training becomes available for each new KU.

1.4 Thesis Structure

The rest of the thesis is organized as follows: In Section 2 we discuss related work on the identification of developer expertise from source-code artifacts/measures. All steps and decisions of the proposed methodology are presented in Section 3.1. Additional improvements we've made to the approach can be found in Section 3.2. Section 4 demonstrates the KU Detection Tool we developed that uses the trained models to detect skills from code artifacts. Section 5 introduces a case study showcasing the potential of ML classifiers for the identification of KUs in GitHub repositories, while Implications to practitioners and researchers are discussed in Section 6. Limitations of the proposed approach and threats to the validity of the case study findings are listed in Section 7. Next, in Section 8 we suggest future improvements regarding both the models and the tool. Finally, we conclude in Section 9.

2 Related Work

Understanding and identifying developer expertise through automated methods is an evolving area of research. Several studies have delved into different methodologies to achieve this aim. From more traditional methods that usually rely on manual analysis, keyword matching and heuristics, to more advanced ones that make use of ML and Natural Language Processing (NLP).

Traditional approaches, while useful, do have their limitations. For example, they may not be able to accurately capture nuanced expertise and can be labor-intensive and time-consuming to create and maintain. Greene and Fischer [9] developed CV-Explorer, a tool that mines GitHub repositories to extract and visualize contributor's technical skills. While effective in providing a broad view of a developer's skill set, this approach requires significant data aggregation and may miss more subtle skills that are not encountered in commit messages or file changes.

Teyton et al. [10] introduced XTic, a tool designed to automatically identify and measure developer skills and expertise levels from source code repositories. XTic uses a domain-specific language (DSL) to define skills and an automatic process to extract these skills from software repositories. The tool validates developer skills and experience levels through syntactical changes developers make in their code and uses clustering algorithms to rank them. Validation of XTic on open-source and industrial projects showed moderate to strong accuracy and good scalability, making it a promising tool for automated expertise identification in software engineering.

Constantinou and Kapitsaki [11] proposed an approach to identify developers' expertise based on their GitHub commit activity. This approach measures developers' commit activity, considering both the quantity and continuity of their contributions over time. Expertise profiles are generated and validated against recognized answering activity in Stack Overflow, showing that the method effectively identifies expertise across different programming languages.

In more recent years, however, many studies have started experimenting with the use of Machine Learning, Neural Networks and Large Language Models (LLMs).

Nguyen et al. [12] investigate the use of LLMs for skill extraction (SE) in the job market domain, a task that involves identifying and extracting specific skills mentioned in job postings and resumes. The authors explore the application of in-context learning capabilities of LLMs to overcome challenges posed by traditional supervised models that rely heavily on manually annotated data. By testing LLMs on a benchmark of six uniformized SE datasets, they demonstrate that while LLMs may not match the performance of traditional supervised models, they excel in handling syntactically complex skill mentions. This research highlights the potential of LLMs in capturing more nuanced skill patterns and addresses the limitations of conventional BIO-tagging approaches. The findings underscore the evolving role of LLMs in enhancing the accuracy and efficiency of skill extraction tasks, which is pivotal for applications such as job matching and market trend analysis.

Nam et al. [13] explored the potential of LLMs to aid in code understanding by developing an IDE plugin called GILT (Generation-based Information-support with LLM Technology). The tool integrates OpenAI's GPT-3.5-turbo model to provide developers with on-demand explanations, details of API calls, domain-specific term definitions, and usage examples. Their study, involving 32 participants, found that the use of GILT significantly improved task completion rates compared to traditional web search methods. However, the study also noted that the benefits varied between students and professionals, with professionals experiencing more significant gains. This research highlights the promise of LLM-based tools in enhancing code comprehension and developer productivity, underscoring the evolving role of advanced AI models in software development environments.

The exceptional ability of LLMs to handle and understand complex syntax led to them being used for code analysis as well. The introduction of the HumanEval dataset by Chen et al. [14] has provided a standardized benchmark for evaluating the functional correctness of code generated by LLMs. HumanEval consists of 164 original programming problems designed to assess language comprehension, algorithms, and simple mathematics through unit tests. Various LLMs have been tested on this dataset, demonstrating their potential in handling code-related tasks. For instance,

OpenAI's Codex, which powers GitHub Copilot, achieved a 28.8% pass rate with a single sample and up to 77.5% with 100 samples. This performance significantly surpasses earlier models such as GPT-3 and GPT-J, highlighting the advancements in LLM capabilities in recent years. Table 1 from OpenAI's GitHub repository[1] further illustrates the competitive performance of state-of-the-art LLMs like GPT-4 and GPT-4-turbo, with pass rates exceeding 90% on HumanEval, underscoring their promise in code understanding and skill extraction applications.

Table 1: Performance of various LLMs on the HumanEval dataset

Model	**HumanEval**
OPENAI GPT-4	90.2
gpt-4o	91.0
gpt-4-turbo-2024-04-09	87.6
gpt-4-turbo-2024-04-09	88.2
gpt-4-1106(-vision)-preview	82.2
gpt-4-1106(-vision)-preview	83.7
gpt-4-0125-preview	88.2
gpt-4-0125-preview	86.6
Claude-3-Opus (rerun w/ api)	84.8
Claude-3-Opus (rerun w/ api)	82.9
Llama3 70b (rerun w/ api)	70.1
Claude-3-Opus (report5)	84.9
Gemini-Ultra-1.0 (report6)	74.4
Gemini-Pro-1.5 (report6)	71.9
Llama3 8b (report7)	62.2
Llama3 70b (report7)	81.7
Llama3 400b (still training, report7)	84.1

From tasks such as locating software defects and detecting AI-generated code to extracting developer skills, LLMs seem to provide quite satisfactory performance.

In a different study, Sahar et al. [15] proposed DP-CCL, a supervised contrastive learning approach that uses the CodeBERT pre-trained model to predict software defects. This method combines the exported CodeBERT semantic features with handmade software metrics to imrpove the classification accuracy. Evaluated on 10 projects from the PROMISE dataset, DP-CCL exceeded current approaches by achieving an in-

[1] https://github.com/openai/simple-evals

crease of 4.9% to 14.9% in the F-1 score metric, and thus proving its effectiveness in detecting software defects.

Russel et al. [16] created a C/C++ source code vulnerability detection system using ML. The authors compiled a dataset of more than 12 million functions from various open source repositories and labeled them using static parsers. They then created a lexer to standardize the code representation and used CNNs and RNNs to extract features from it. Combining those features using ensemble classifiers (Random Forest and Extra Trees) showed improvement in detection accuracy. Their approach significantly outperformed existing methods, which goes to show the potential of using deep learning for automated vulnerability detection.

Li et al. [17] presented DP-CNN, a framework for software defect prediction that uses deep learning. They leverage CNNs to automatically generate semantic and structural features from source code ASTs (Abstract Syntax Trees) and combine them with hand-crafted features for more accurate predictions. DP-CNN demonstrated a 12% improvement compared to the leading Deep Belief Network (DBN) method and a 16% improvement over traditional feature-based methods.

Nguyen et al. [18] developed GPTSniffer, a tool that uses CodeBERT to detect source code generated by ChatGPT. By evaluating it on datasets with code from GitHub and ChatGPT, this tool significantly surpasses existing classifiers like GPTZero and OpenAI's Text Classifier. Possible areas of application include education where it can be used to detect plagiarism and during development to ensure the quality of AI generated code.

Kourtzanidis et al. [19] utilized Microsoft's Language Understanding Intelligent Service (LUIS) to refine a model capable of detecting three specific technologies within the .NET ecosystem, as well as two other popular frameworks (React and Angular). LUIS yielded results of very high accuracy (F-measure > 0.9) despite the limited number of utterances used for training the model.

Another investigation [20] aiming at visualizing developer expertise applied techniques from the Natural Language Processing (NLP) domain. This work utilized LDA and LLDA topic models on source code activities to infer areas of expertise. The

approach involved preprocessing commit histories and applying topic modeling to discern latent knowledge patterns in the code. A discrete skill level between one and five is assigned to each developer, based on the use of relevant words in his/her commits.

The topic of reviewer selection is highly relevant to skill identification, as the goal is to find individuals with expertise that matches the technologies used in an artifact under review. Ye et al. [21] introduced a method to suggest suitable reviewers for pull requests. Their approach encoded various pull request components—title, commit message, and code changes—into semantic vectors. This encoding utilized LSTM-networks for textual elements and CNNs for code, subsequently predicting optimal reviewers.

Cury et al. [22] employed repository-based metrics to discern correlations with developer expertise. Key findings highlighted that First Authorship and the Number of Lines Added were the most positively correlated metrics. Leveraging machine learning models like Random Forest, SVM, and Gradient Boosting Machines, this study surpassed traditional methodologies in pinpointing source code experts using version control data.

Moving one step further, Montandon et al. [23] proposed an unsupervised approach to single out software library experts. Their methodology hinged on clustering feature data extracted from GitHub, with subsequent triangulation of results against expertise data from LinkedIn.

3 Methodology

3.1 Approach and Implementation

3.1.1 Definition of Knowledge Units

Our study leveraged the classification of KUs and their corresponding Key Capabilities (KCs) presented in [2], which were based on Oracle's Java certification exam topics. We adapted this framework, as shown in Table 7, by omitting certain KUs based on practical challenges faced during our research. Specifically, the declaration and initialization of variables (K1) was so fundamental that sourcing negative examples without such code in sufficient quantities for the binary classifiers became challenging. Furthermore, when attempts were made to train models for both K1 and K5, it became evident that, while feasible, assessing their real-world performance using the sliding window technique posed significant difficulties. This was especially the case for K5 (Methods and Encapsulation), where foundational elements like the use of the keyword "static" or the inclusion of access modifiers are intrinsic to Java, complicating the process of finding negative examples suitable for accurately training our ML models. Additionally, Java Server Faces (K26) largely intertwines with XML files, which serve to define and structure its data. Despite instructing ChatGPT to generate only Java files associated with Java Server Faces, it predominantly produced XML files. This trend made it impractical to obtain the desired necessary volume of Java-related code snippets, leading us to exclude K26 from our study.

3.1.2 Dataset Creation

To ensure comprehensive representation of each KC, we amassed 1,000 code snippets per KC. Given that manually curating approximately 90,000 code snippets, particularly for the rarer KUs, would be a monumental task, we employed ChatGPT (GPT-3.5) for custom snippet generation. Despite GPT-4 being a more capable model, its speed was a significant limiting factor and therefore GPT-3.5 was chosen instead. To counteract potential hallucinations and irrelevant code snippet generation, we tasked ChatGPT with composing the code snippets in batches, given the following prompt:

"Write [number of snippets] code snippets in Java related to [Knowledge Unit]. More specifically, these snippets should be directly related to the following key capability: [Key Capability]. Try covering a wide range of examples. Each code snippet should be in a different code block. Do NOT include usage examples with the code you create."

Minor alterations to the prompt were made in instances where ChatGPT deviated from the provided guidelines. In Fig. 1 you can see a code snippet about WebSockets generated by ChatGPT. Normally, code snippets like this one would require an expert in this domain to manually write it or find it online and inspect which parts are relevant and which are not.

Subsequent to the initial dataset generation, refinement was necessary. Since a complete manual review of snippets was impractical, our methodology entailed devising a keyword list for each KC within a KU. We then screened snippets for absence of these keywords. While in other contexts, relying on keyword detection may pose challenges due to potential overlap of keywords across different KCs (e.g. common method names), this wasn't a concern in this case. Given that ChatGPT was explicitly directed to generate snippets for a specified KC, the majority of the produced snippets were appropriately aligned with that KC. As such, detected keywords were utilized in the context that was specific to the KC in question. Predominantly, these keywords encompassed reserved words, specific class and method names, symbols, and decorators relevant to the respective KC. Flagged snippets (those in which no word from the keyword list was detected) constituted a minor segment of the dataset, allowing manual inspection and necessary replacements. This refinement process was reiterated until the dataset met our standards.

3.1.3 Code Preprocessing

Code snippets were then processed to ensure they conformed to a format conducive to model training. Steps included stripping comments and blank lines, replacing strings, characters, numbers, and booleans with placeholders to generalize the dataset and mitigate biases from specific values ChatGPT might favor during snippet generation, and subsequently tokenizing the code. This approach emphasizes the code's

```java
import javax.websocket.*;
import javax.websocket.server.ServerEndpoint;
import java.io.IOException;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentMap;

@ServerEndpoint("/authorized")
public class AuthorizedWebSocketServer {

  private static final String AUTHORIZATION_HEADER = "Authorization";
  private static final ConcurrentMap <String, String> authorizedUsers =
    new ConcurrentHashMap<>();

  @OnOpen
  public void onOpen(
    Session session,
    EndpointConfig config
  ) throws IOException {
    String token =
      (String) config.getUserProperties().get(AUTHORIZATION_HEADER);
    if (isAuthorized(token)) {
      authorizedUsers.put(session.getId(), token);
      // Handle authorized connection initiation
    } else {
      session.close(
        new CloseReason(
          CloseReason.CloseCodes.CANNOT_ACCEPT,
          "Unauthorized."
        )
      );
    }
  }

  @OnMessage
  public void onMessage(String message, Session session) {
    // Handle authorized message
  }

  @OnClose
  public void onClose(Session session, CloseReason reason) {
    authorizedUsers.remove(session.getId());
  }

  private boolean isAuthorized(String token) {
    // Implement your token validation logic
    return true;
  }
}
```

Figure 1: Code snippet generated using ChatGPT for the WebSockets KU (K25)

structural and syntactic features, steering the model's focus away from particular values and towards the overarching patterns of KUs and KCs.

3.1.4 Training and Validation

Given the variability in the number of KCs within KUs (ranging from 1 to 8), a primary challenge was the determination of sample sizes per KU or KC. Although feasible, training a singular multi-label classification model posed potential performance and training challenges for the binary classifiers, particularly due to the presence of 27 discrete classes. Consequently, we opted for 27^2 distinct binary classifiers via a one-vs-rest strategy. Each KC was represented with 1,000 code snippets. When training a model for a specific KU, all samples from that category were designated as the positive class. An equivalent number of samples were aggregated from the other KUs, serving as the negative class. When selecting samples for the negative class, we made sure to select an equal number from each KU, and when multiple KCs were present in a KU, an equal number of samples were picked from each KC as well.

After sub-sampling from the dataset, we tokenized and subsequently vectorized the code snippets before feeding them into the models to begin the training. An overview of the entire training process can be found in Fig. 2

3.1.5 Model Selection

Our quest for the optimal model for each KU entailed the implementation of k-fold cross validation across various classifiers, which included Multilayer Perceptron (MLP), Multinomial Naive Bayes (MNB), Support Vector Machine (SVM), Logistic Regression (LR), Random Forest (RF), k-Nearest Neighbors (kNN), and Gradient Boosting (BG). Preliminary results (showcased in Table 2 and Table 3) underscored the exemplary performance of MLPs, SVMs, and RFs.

Despite superior performance in certain categories, Random Forest classifiers were not selected to be used for the analysis tool. The extended prediction times—especially with a substantial volume of prediction tasks—slowed down and compromised the

[2]Due to insufficient relevant samples, no model was trained for K26. For K1 and K5, however, despite the challenges mentioned in 3.1.1, models were trained and later deemed unfit to be used.

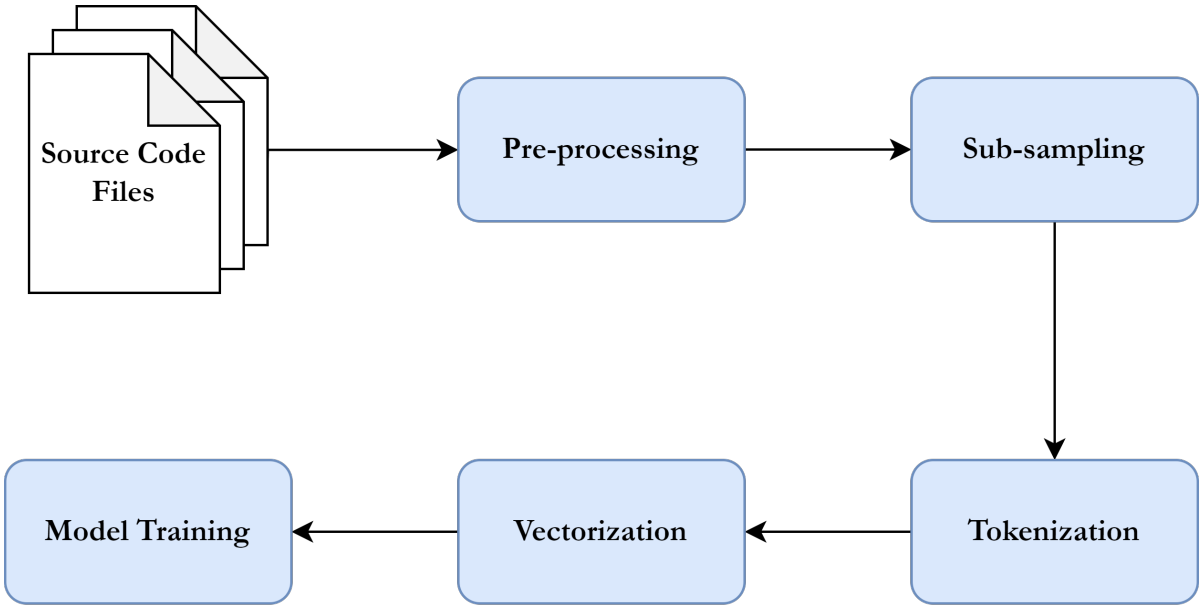

Figure 2: Binary classifiers training process

overall efficiency of the analysis process.

3.1.6 Skill Identification - Sliding Window Technique

The models were trained and ready to make predictions, but they needed to be given code snippets, not entire code files. For instance, if a 3-line code snippet was embedded within a 200-line file or a 15-line snippet within a 600-line file, directly inputting the entire file might obscure the targeted KU detection. To address this, we applied the sliding window technique, similar to the one used for image detection in computer vision.

The process begins with a minimum window size, scanning the file from start to finish, capturing the code segment within the window during each pass and feeding it into the model to check if there is a detection. Upon completing the sweep with a particular window size, it is incremented, and the process is reiterated until the window reaches the designated maximum size or a detection is made. This ensures a comprehensive extraction of code snippets, spanning diverse lengths, thereby enhancing the likelihood of suitably aligning a KU within a window, optimizing the model's detection capability. This process is repeated for each binary classifier.

Apart from the minimum and maximum window sizes, there are 2 more parameters

Table 2: Accuracy of the examined classifiers for each KU (all values represent percentages)

KUs	MLP	MNB	SVM	LR	RF	kNN	GB
K2	98.25	94.81	98.44	97.94	98.62	96.44	98.06
K3	96.00	93.00	96.62	94.75	95.25	94.75	94.62
K4	97.15	92.70	97.10	96.15	98.05	94.70	98.05
K6	97.13	95.59	97.21	96.25	96.59	95.21	95.88
K7	96.46	93.36	97.95	96.71	98.08	91.81	97.83
K8	96.08	94.33	96.76	95.64	95.39	93.21	94.15
K9	97.00	92.88	98.19	96.19	97.56	92.75	99.25
K10	98.61	96.64	98.75	98.57	98.68	95.71	98.36
K11	96.25	94.00	96.61	96.04	96.93	84.58	96.32
K12	99.57	99.51	99.45	99.45	99.20	99.51	98.65
K13	96.25	95.00	96.00	95.75	96.75	92.88	95.75
K14	99.25	98.38	99.50	99.62	99.62	99.00	98.88
K15	97.87	96.31	98.56	97.62	97.81	95.75	97.56
K16	99.19	98.50	99.50	99.12	99.00	98.12	97.62
K17	97.75	98.00	97.38	97.38	96.75	96.88	96.00
K18	98.75	98.12	99.12	98.38	99.00	97.50	98.75
K19	99.67	98.17	99.67	98.92	99.58	98.58	99.08
K20	98.00	97.50	99.12	97.75	98.88	95.62	98.12
K21	99.62	98.62	99.50	98.75	99.25	98.38	98.88
K22	98.37	98.62	99.00	98.00	98.75	96.88	97.62
K23	99.17	98.75	99.42	98.92	99.25	98.08	99.08
K24	99.37	98.50	99.62	98.88	99.38	99.12	99.00
K25	98.50	97.12	98.62	97.25	98.00	97.25	97.62
K27	97.25	96.50	97.50	96.75	96.50	96.75	98.50
K28	100.00	96.75	99.25	98.00	99.75	99.25	99.75

Table 3: Loss of the examined classifiers for each KU

KUs	MLP	MNB	SVM	LR	RF	kNN	GB
K2	0.048	0.137	0.043	0.155	0.059	0.505	0.066
K3	0.114	0.166	0.110	0.200	0.135	0.629	0.161
K4	0.077	0.215	0.080	0.144	0.063	0.709	0.064
K6	0.089	0.134	0.088	0.134	0.098	0.605	0.115
K7	0.097	0.183	0.100	0.169	0.095	0.899	0.094
K8	0.118	0.187	0.113	0.207	0.170	1.073	0.171
K9	0.074	0.170	0.050	0.181	0.151	0.591	0.066
K10	0.042	0.118	0.047	0.094	0.055	0.421	0.071
K11	0.105	0.180	0.113	0.166	0.140	2.368	0.127
K12	0.019	0.032	0.025	0.081	0.051	0.032	0.061
K13	0.095	0.141	0.130	0.180	0.095	0.499	0.118
K14	0.022	0.044	0.010	0.087	0.022	0.070	0.065
K15	0.058	0.100	0.048	0.125	0.083	0.415	0.078
K16	0.270	0.061	0.020	0.118	0.067	0.125	0.079
K17	0.070	0.080	0.084	0.164	0.109	0.449	0.124
K18	0.040	0.064	0.038	0.124	0.063	0.222	0.070
K19	0.018	0.059	0.020	0.079	0.029	0.203	0.047
K20	0.066	0.102	0.064	0.122	0.064	0.426	0.068
K21	0.028	0.065	0.050	0.089	0.071	0.122	0.050
K22	0.054	0.079	0.053	0.158	0.072	0.192	0.083
K23	0.032	0.056	0.027	0.118	0.051	0.207	0.048
K24	0.011	0.056	0.010	0.095	0.029	0.028	0.039
K25	0.040	0.080	0.035	0.140	0.071	0.274	0.081
K27	0.062	0.115	0.112	0.206	0.108	0.094	0.052
K28	0.005	0.086	0.026	0.114	0.023	0.121	0.038

that need to be set. The first one being the ***window increase step***, which determines how much the height of the window (i.e. the number of lines it covers) increases between sweeps. The second one is the ***move step*** which controls the distance between subsequent windows (e.g. a window could begin 1 line below the previous window or 10 lines below it to reduce overlap). Given the algorithm's quadratic complexity, the values used for the 4 aforementioned parameters play a critical role on the prediction times and detection accuracy. Therefore it's important to find values that optimize performance while minimizing computational iterations.

3.1.7 Commit History Analysis

Examining the commit history in a git repository extracts useful insights into the technologies a project employs and the expertise distribution among contributors. Such insights can illuminate strengths and gaps in a developer's grasp of a programming language (Java, in this context) and guide hiring decisions, as discussed in Section 1.

In our methodology, we operate under the assumption that when a developer makes specific changes to a code file, they possess comprehensive understanding of the entire file's content. To attribute KU detection accurately to the respective coder, only non-merge commits were considered, ensuring the commit's author genuinely reflected the code changes.

For each modified or newly created Java file per commit (excluding those with only comment or import adjustments), we probed its contents for KU presence. In the results exported from the algorithm we documented the detected KUs, the commit author, and the timestamp and SHA of the commit. Using this data, we crafted charts illustrating knowledge distribution and delineating interrelations between KUs. These insights, complemented by a case study, are elaborated upon in Section 5.

3.1.8 KU Detection Algorithm Evaluation

We gauged the performance of the sliding window approach using 22,981 code files from 22 GitHub repositories. Subsets of this dataset, containing a balanced number of positive and negative samples, were employed to evaluate each model. Given that

manually tagging each sample with all its pertinent KUs was infeasible, we turned to regular expressions for labeling true occurrences. For example, `\btry\s*\{` was used to detect try statements. Although these regular expressions utilized keywords similar to those in Section 3.1.2, their application was more stringent. In this context, it was crucial to be more precise, as the potential overlap of keywords across multiple KUs could introduce ambiguity—a stark contrast to Section 3.1.2, where such overlap wasn't problematic. The outcomes of this evaluation strategy are detailed in Table 4.

Table 4: Evaluation of the sliding window technique for each KU (all values represent percentages)

KUs	Accuracy	Precision	Recall	F1 Score
K2	83.05	81.63	85.30	83.42
K3	70.15	66.67	80.60	72.97
K4	73.50	100.00	47.00	63.95
K6	60.20	55.86	97.30	70.97
K7	61.40	56.43	100.00	72.15
K8	58.45	54.80	96.40	69.88
K9	93.25	91.95	94.80	93.35
K10	88.25	90.14	85.90	87.97
K11	69.55	62.15	100.00	76.66
K12	81.88	80.97	83.33	82.14
K13	76.75	68.85	97.70	80.78
K14	92.25	89.85	95.25	92.47
K15	72.45	67.23	87.60	76.07
K16	72.35	64.77	98.00	77.99
K17	66.83	60.20	99.31	74.96
K18	76.01	68.11	97.80	80.30
K19	81.07	72.95	98.77	83.92
K20	57.17	54.29	90.84	67.96
K21	85.25	79.89	94.20	86.46
K22	61.29	56.36	100.00	72.09
K23	64.00	60.77	79.00	68.70
K24	64.04	62.35	70.88	66.34
K25	59.35	55.82	89.70	68.81
K27	73.46	71.21	78.77	74.80
K28	56.30	54.50	76.30	63.58

3.2 Further Improvements

After creating a first working version of the above proposed system, further experimentation was conducted in order to improve the system even more. This included: re-labeling the dataset to support the training of multi-label classification models, removing duplicate samples and fine-tuning CodeBERT (a pre-trained LLM specific to programming languages) to test its performance on the given task of identifying KUs in code snippets.

3.2.1 Dataset Improvement

One limiting factor of the original dataset was that each sample was labeled with a single label; the KU that ChatGPT was instructed to generate code for. That, however, is not always accurate, as KUs tend to overlap with each other. For example, when generating the code snippet from Fig. 3 as a positive sample for K10 (Stream API), we end up having other KUs present as well. Apart from Stream API-related methods, we can also see the use of:

- Data Types (K1) when declaring and initializing the **`numbers`** variable
- Operators and Decisions (K2) when using the assignment operator (**`=`**)
- Generics and Collections (K8) when creating an Arrays list and iterating over it with **`.forEach`**
- IO (K13) when printing the contents of the list to the console

```java
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);

numbers.stream()
    .peek(System.out::println)
    .forEach(e -> {});
```

Figure 3: Code snippet generated using ChatGPT for the Stream API KU (K10)

Therefore, re-labeling the dataset became an important step before being able to use it for accurately training any multi-label classification model. For this task, regular expressions were employed once again, while also making sure to only look for method names and keywords that were specific to a single KU at a time and not shared across multiple ones.

Furthermore, an analysis revealed that approximately 14.9% of the code snippets in the original dataset were recurrent, leading to potential data redundancy. Many samples had identical code structure with the only difference between them being the values of variables. After pre-processing the code samples, which replaced all variable values with placeholder text, the duplicates percentage rose to 21.09%. Such redundancy, depending on its distribution across KU categories, could unevenly influence the performance of our ML models across these categories. Depending on how many duplicate samples remain after sub-sampling the dataset to train the binary classifiers, some categories might show artificially inflated performance due to repeated exposure to identical snippets, while others might remain unaffected. To achieve a more precise and representative measure of the models' performance, it would be advisable to eliminate these redundant snippets and supplement the dataset with new ones.

This proved to be challenging for the rarest KUs, such as K27 and K28, where ChatGPT would often start repeating itself. Many iterations were required to generate new unique samples and even then they would oftentimes be quite similar with minimal alterations between them. In the case of K28, when we couldn't generate any more unique code snippets and the total number of samples was still lower than the 1,000 it used to be, we were left with 2 choices: either select fewer counterexamples to keep the training set balanced but risk being underfit due to limited training data or have more negative samples and risk the model becoming biased.

This pushed us to start experimenting with neural networks (NNs) and more specifically with CodeBERT, which was pre-trained so it could handle the limited samples more easily without sacrificing accuracy and performance. After assigning multiple labels to each sample to represent all KUs that were present in it and completely re-

moving any duplicates, the distribution of samples significantly changed as we can see in Table 5.

Table 5: Comparison of number of positive samples per KU between the original dataset and the updated one (after re-labeling and removing duplicates)

KUs	Single-label dataset	Multi-label dataset
K1	1,017	49,712
K2	4,000	59,361
K3	2,000	7,304
K4	5,000	9,413
K5	8,017	28,028
K6	6,004	12,339
K7	4,030	6,085
K8	4,013	19,558
K9	4,000	4,365
K10	7,000	7,597
K11	7,003	9,823
K12	4,058	3,907
K13	2,000	13,495
K14	2,000	2,496
K15	4,000	3,845
K16	4,000	4,287
K17	2,000	2,095
K18	2,000	1,947
K19	3,000	2,549
K20	2,000	2,698
K21	2,000	1,658
K22	2,000	1,707
K23	3,002	1,779
K24	2,000	1,860
K25	2,000	1,771
K27	1,000	943
K28	1,000	879
Total number of files	90,144	71,135

Some KUs saw tremendous increase in positive samples, like K1 (Data Types) and K2 (Operators and Decisions) which are fundamental building blocks of many programming languages. K5 (Methods and Encapsulation), K6 (Inheritence) and K8 (Generics and Collections) are quite common building blocks in Java—given that it's an object-oriented programming language—and they saw a considerable increase in

samples as well. K13 (IO) positive samples increased by more than 11,000 mostly because when ChatGPT generates example code, it usually tends to print the methods' results or variables' values to the console.

More advanced KUs (i.e. those that are part of Java Enterprise Edition) lost samples due to the removal of duplicates but didn't manage to substitute them with samples from other categories. That is to be expected, as more basic KUs such as K1, K2, K3 and K4 are more likely to be generated by ChatGPT as part of a different KU's code snippet. While on the other hand, more advanced KUs like K27 and K28 will not be randomly generated unless we specifically ask for them.

3.2.2 CodeBERT

CodeBERT is a pre-trained model based on the RoBERTa architecture [24]. It consists of a multi-layer transformer network with 125 million parameters but only uses the encoder part of the transformer, as seen in Fig. 4. The encoder is made up of 6 identical layers. It has been trained on 6 programming languages: Python, Java, JavaScript, PHP, Ruby and Go, making it ideal for our case of examining Java code.

We followed the same code pre-processing as with the binary classifiers and then used the new cleaned multi-label dataset to fine-tune CodeBERT. Fig. 5 depicts the steps taken to fine-tune CodeBERT. It is similar to Fig. 2 with the only difference being that for CodeBERT the entire dataset was used, instead of only a subset of it. Despite limited Google Colab resources, we were able to train the model for 5 epochs and the results already surpassed all 7 binary classifiers we used before in terms of accuracy, as seen in Table 6.

Ideally, we would want to fine-tune CodeBERT for more than 5 epochs in order to see the true capabilities and performance of the model. However, even with this limited training the result were quite promising. Looking at the loss curves diagram in Fig. 6 we can see both training and validation losses decrease consistently. This indicates that the model is learning effectively. The relatively small gap between the two curves suggests that the model generalizes well to unseen data and is not overfitting.

This was a significant step towards improving the efficiency of the KU Detection

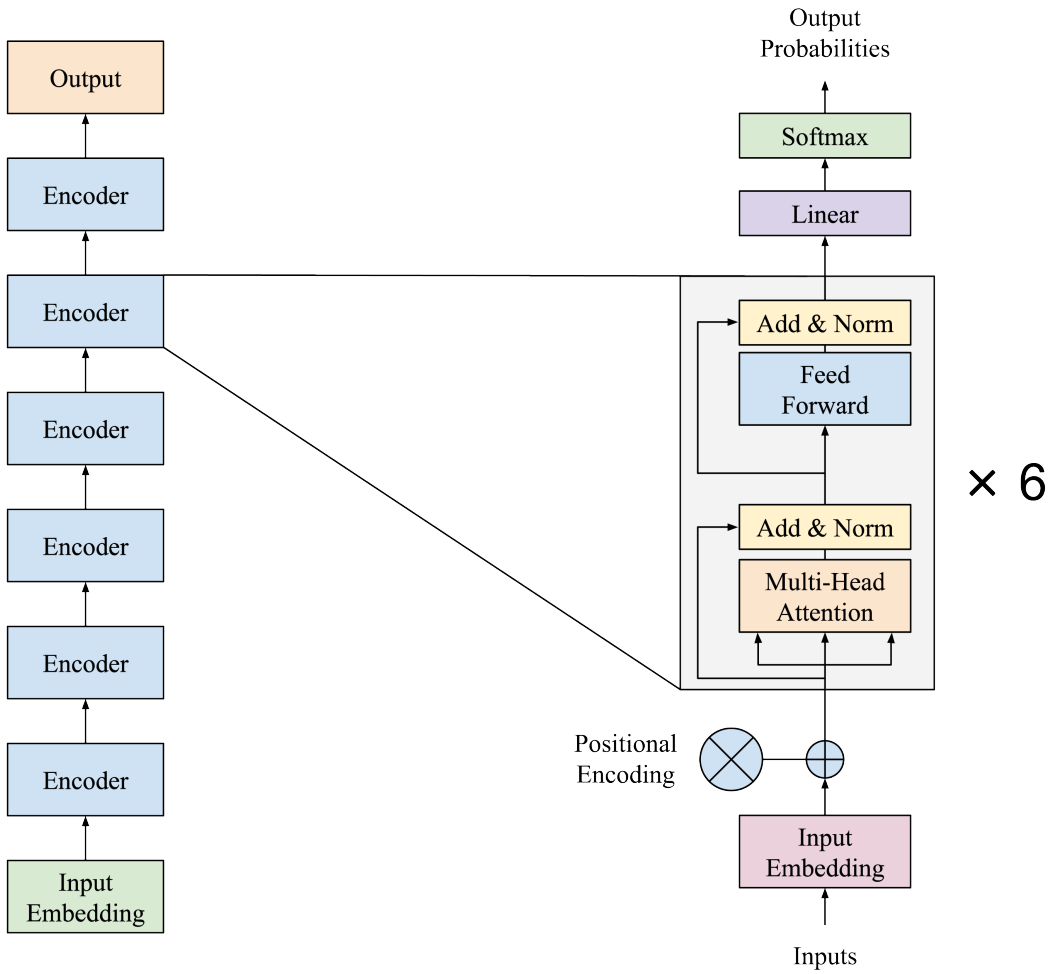

Figure 4: CodeBERT model architecture (adapted from [1])

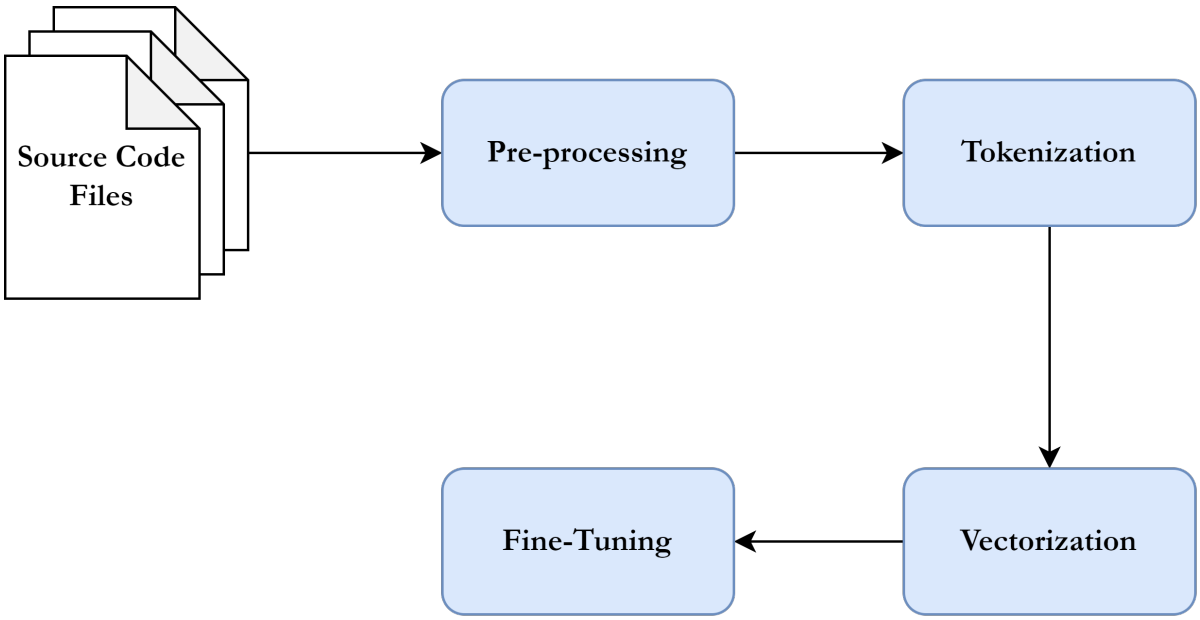

Figure 5: CodeBERT fine-tuning process

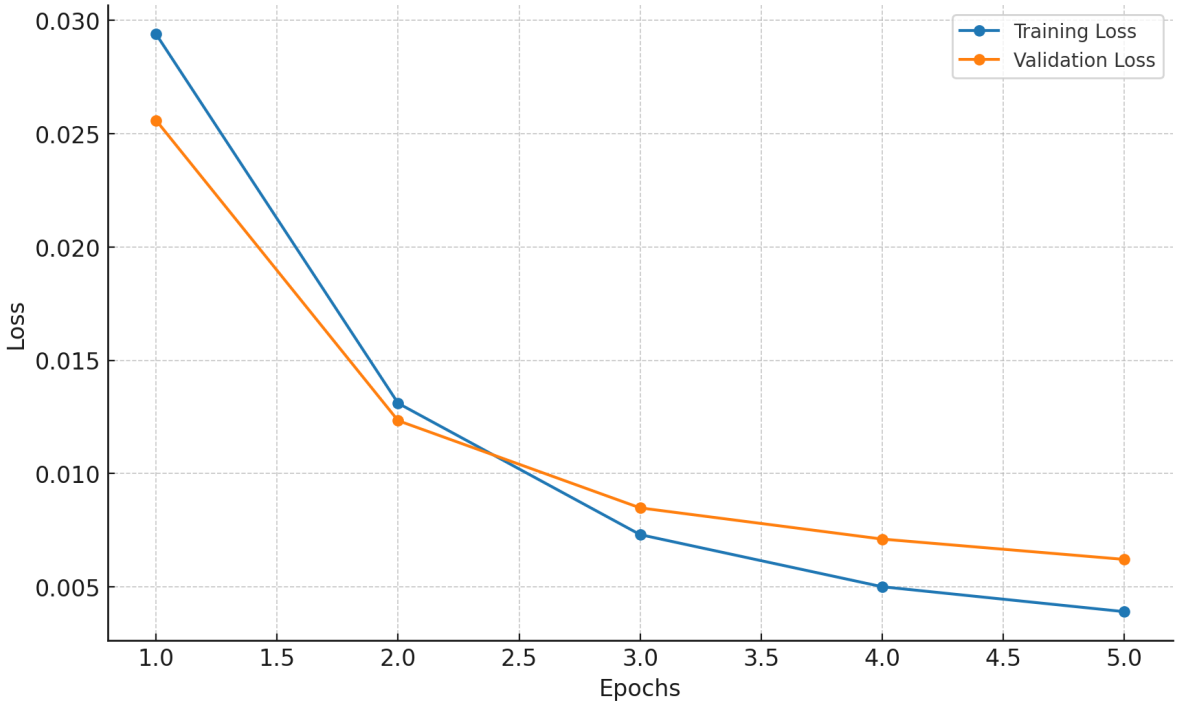

Figure 6: CodeBERT fine-tuning loss curves

tool as we'll discuss in section 4.

A replication package including: a) the old and new datasets with all Java files used for training the classifiers, b) a Jupyter notebook for training ML-models using cross-validation, c) a Jupyter notebook for fine-tuning CodeBERT d) the tool for scanning git repositories and extracting KUs (both front-end and back-end), e) all the trained models and f) the dataset with all the Java files and Python scripts used for evaluating the sliding window technique's performance is available online [3].

[3] https://zenodo.org/records/10056038

Table 6: Evaluation of the fine-tuned CodeBERT model (all values represent percentages)

KUs	Accuracy	Precision	Recall	F1 Score
K1	99.76	99.85	99.80	99.83
K2	99.96	99.97	99.98	99.98
K3	99.89	99.63	99.34	99.48
K4	99.96	100.00	99.70	99.85
K5	99.84	99.92	99.69	99.80
K6	99.58	99.32	98.25	98.78
K7	99.82	99.16	98.69	98.92
K8	99.87	99.92	99.61	99.76
K9	99.91	98.81	99.76	99.28
K10	99.89	99.34	99.63	99.49
K11	99.70	99.00	98.83	98.92
K12	99.90	99.14	99.00	99.07
K13	99.82	99.55	99.51	99.53
K14	99.98	99.56	99.78	99.67
K15	99.72	97.56	97.28	97.42
K16	99.89	98.55	99.60	99.07
K17	99.81	98.44	95.45	96.92
K18	99.95	99.39	98.79	99.09
K19	99.88	98.76	98.15	98.46
K20	99.90	98.89	98.24	98.56
K21	99.97	99.33	99.33	99.33
K22	99.94	99.36	98.11	98.73
K23	99.95	98.82	99.11	98.97
K24	99.85	97.65	96.80	97.23
K25	99.97	100.00	98.73	99.36
K27	99.84	96.41	91.48	93.88
K28	100.00	100.00	100.00	100.00

4 KU Detection Tool

The goal of this thesis was not only to train models for KU detection, but also to build a tool that would utilize those models and enable end users to easily interact with Git repositories, analyze their commits and gain valuable insights about the skills distribution among the contributors of each project.

The tool is split into 2 parts:

(a) The back-end which contains all the logic and is responsible for handling Git operations, analyzing files and using the trained models to make predictions about the KUs that are present within those files, and providing an API endpoint to allow integration with other systems.

(b) The front-end that provides a user-friendly UI which allows users to communicate with the back-end via API and view the analysis results in an interactive environment.

4.1 Back-end

The back-end[4] was developed using Python for its ease of use and abundant community support. The `Transformers`, `TensorFlow` and `PyTorch` packages were used for loading and using the trained models.

Even though the tool is a demo and provides limited functionality, it's been developed with possible future expansions in mind. The code has been grouped into relevant modules and it follows the file structure of Fig. 9. It's explained more in-depth below.

4.1.1 `api/` directory

The `api/` route handles all the API-related logic, allowing external systems to communicate and interact with the rest of the back-end system. Below is a breakdown of what each of its files contains.

[4]The source code can be found on GitHub here `https://github.com/NikolasPpd/KU-Detection-Back-End`

`main.py`: Initializes a simple Flask app and listens to the specified port for API requests.

`routes.py`: Contains the following 2 routes and handles API requests:

`/commits`: Accepts `repo_url` which is a Git repository URL and an optional `limit` parameter to limit the number of commits to be analyzed. If the repository doesn't already exist on the server, it is first cloned. Then the specified number of contributions is extracted, cached on the server and sent as a response in JSON format. An example of the JSON response can be found in Fig 7. The response is an array of objects. Each object corresponds to a single modified file within a commit and has the following fields:

- **`author`:** The username of the person who modified the file.
- **`changed_lines`:** Numbers of modified lines extracted from the file diff. Currently not being used in any way but will be important information when we incorporate code slicing into the workflow (more details about code slicing in Section 8).
- **`file_content`:** The contents of the file after it was modified.
- **`sha`:** The SHA of the commit that contained the modified file.
- **`temp_filepath`:** The path where the modified file is temporarily saved on the server.
- **`timestamp`:** Timestamp of when the commit was created.

`/analyze`: Accepts `repo_url`, retrieves cached contributions data, sends them for analysis and returns the results incrementally. This means that we don't wait for the entire analysis to finish and send all the results at once at the end, which could take a considerable amount of time, depending on the amount of files being analyzed. Instead, we send them one by one as they are being produced. This provides a constant

```
[
    {
        "author": "alice",
        "changed_lines": [
            321,
            324,
            325,
            327,
            328,
            329,
            331,
            332,
            345
        ],
        "file_content": "...",
        "sha": "4be8594bdefa3b941d062cbcee0794a3718c58b1",
        "temp_filepath": "C:\\KU-Detection-Back-End\\temp\\example_file_name_4be8594.java",
        "timestamp": "2024-06-20T04:41:36"
    },
    {
        "author": "bob",
        "changed_lines": [
            12,
            13,
            14,
            258,
            259
        ],
        "file_content": "...",
        "sha": "2cbcee0794a3718c58b14be8594bdefa3b941d06",
        "temp_filepath": "C:\\KU-Detection-Back-End\\temp\\example_file_name_2cbcee0.java",
        "timestamp": "2024-06-20T04:41:36"
    },
]
```


Figure 7: Mock response sent from `/commits` API endpoint

feedback loop to the user, making it clear that even though it might take some time, the tool is working as expected.

Fig. 8 shows an example of what a single file's results API response looks like. It includes the following fields:

- **`author`:** The username of the person who modified the file.
- **`detected_kus`:** A key-value pair object, where keys are the names of KUs and values are binary flags denoting whether a KU was detected in the file or not (with 1 and 0 respectively).
- **`filename`:** The name of the analyzed file.
- **`sha`:** The SHA of the commit that contained the analyzed file.
- **`timestamp`:** Timestamp of when the commit was created.
- **`elapsed_time`:** How much time it took (in seconds) to analyze the file.

4.1.2 `cloned_repos/` directory

A directory for creating sub-folders with unique names in which repositories will be cloned. This should prevent any conflicts between repositories with identical names when multiple users try to use the tool at the same time. Additionally it should keep the cloned repositories private and only allow access to the user that cloned them. For the purposed of the demo only one folder exists and all repositories are cloned in it.

4.1.3 `config/` directory

Contains `settings.py` which holds global configuration settings for the tool.

4.1.4 `core/` directory

The core functionality of the tool is within this folder. Split into 4 sub-folders that are responsible for: analysis, Git operations, ML operations, and utilities.

```
{
    "author": "alice",
    "detected_kus": {
        "K1": 1,
        "K10": 0,
        "K11": 1,
        "K12": 0,
        "K13": 0,
        "K14": 0,
        "K15": 0,
        "K16": 1,
        "K17": 0,
        "K18": 0,
        "K19": 0,
        "K2": 1,
        "K20": 0,
        "K21": 0,
        "K22": 0,
        "K23": 0,
        "K24": 0,
        "K25": 0,
        "K26": 0,
        "K27": 0,
        "K3": 1,
        "K4": 1,
        "K5": 1,
        "K6": 1,
        "K7": 1,
        "K8": 1,
        "K9": 0
    },
    "filename": "example_file_name_4be8594",
    "sha": "4be8594bdefa3b941d062cbcee0794a3718c58b1",
    "timestamp": "2024-06-20T04:41:36",
    "elapsed_time": 0.16295123100281
}
```


Figure 8: Mock response sent from `/analyze` API endpoint

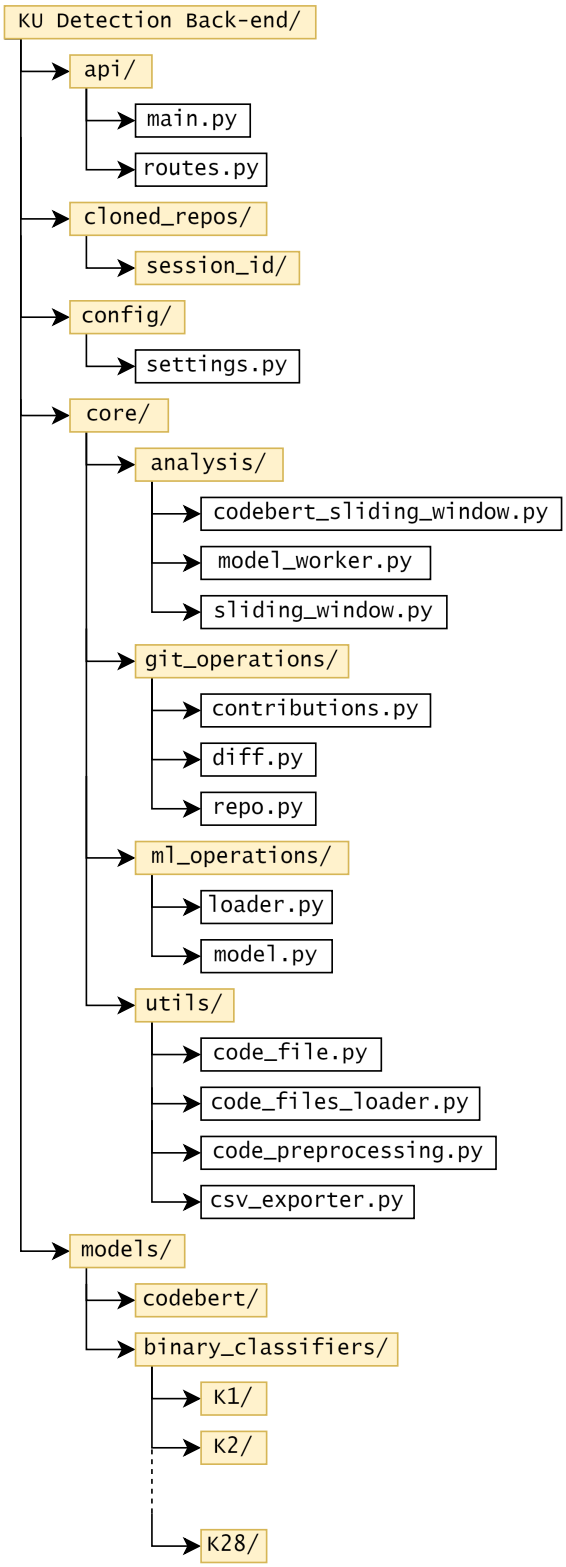

Figure 9: Back-end project structure

4.1.5 `core/analysis/` directory

The `core/analysis/` directory contains the files that implement the sliding window technique and invoke the trained ML models to make predictions about KUs.

`codebert_sliding_window.py`: Implements the sliding window technique and uses CodeBERT to make predictions. Accepts the following arguments:

- **`files`:** A list of files to be analyzed (each file is an object of type `CodeFile`).
- **`min_win_size`:** The minimum size of the window, measured in lines. This is the initial window size.
- **`max_win_size`:** The maximum size of the window, measured in lines. This is the final window size, after which the scanning ends.
- **`win_increase_step`:** How much should the window size increase between iterations. Measured in lines.
- **`move_step`:** How many lines below the previous window of the same size should the current window be moved. When the window size is increased, the window starts from the top of the file.
- **`model`:** Which model to use for predictions. This implementation works with the CodeBERT model.

`model_worker.py`: The first implementation of the sliding window, which works with the binary classifiers. It accepts the same arguments as the CodeBERT sliding window implementation.

`sliding_window.py`: Accepts the same arguments as the CodeBERT sliding window implementation but instead of a single model it accepts a list of them. The key difference is that it uses `ProcessPoolExecutor()` to start multiple processes, load each model to a different process and make predictions concurrently. Initializing

the processes has an overhead but the extra time is minimal compared to the time saved while analyzing large quantities of files.

4.1.6 core/git_operations/ directory

The `core/git_operations/` directory contains all the necessary logic to interact with Git.

contributions.py: Implements `extract_contributions()` which iterates over the specified commits of a given repository, gathers the contributions that were made and returns them in the format shown in Fig. 7.

diff.py: Implements `get_contributions_from_diffs()` which is responsible for analyzing commits. For every modified file in the commit, the file diffs are parsed to help locate the modified lines. Contribution objects are created and returned to `extract_contributions()` which aggregates all those objects before returning them to the API endpoint that initially called the method.

repo.py: Contains various methods that are related to Git repositories. More specifically, there are methods to check if a repository exists locally, clone a repository from a given URL, get a reference to a local repository, get a list of the names of all local branches in the given repository and get a list of the names of all branches in the given repository, including remote branches.

4.1.7 core/ml_operations/ directory

The `core/ml_operations/` directory contains all the necessary logic to load and use the trained models.

loader.py: Contains methods that load the trained models in order to then be used for predictions.

model.py: Contains the `Model` and `CodeBERTModel` classes. These classes implement the `predict()` method. It accepts a Java code string and makes the

necessary pre-processing before making a prediction and returing the detected KUs. Having those classes allows us to group all the relevant logic into one place. Then we only have to load a model and use the object's `predict()` method without having to worry about pre-processing the code, tokenizing it, vectorizing it and turning the raw outputs into binary flags that show if a KU was present in the provided sample or not.

4.1.8 `core/utils/` directory

Various utility functions are gathered in this directory.

`code_file.py`: Implements the `CodeFile` class. Each file is loaded into a `CodeFile` object before being fed into the models. This class groups useful fields and methods related to files, such as the file name, contents, number of lines, and information about the commit in which the file was found.

`code_files_loader.py`: Contains methods that read files and create `CodeFile` objects that are returned to the caller function.

`code_preprocessing.py`: The methods in this file are responsible for removing imports, packages, comments and blank lines, replacing strings, characters, booleans and numbers with placeholder text as well as breaking the code into a list of tokens.

`csv_exporter.py`: Implements a method that exports the file information and the KUs that were detected in it into a CSV file. That file can then be used for further analysis.

4.1.9 `models/` directory

Contains all the trained models. Currently has 2 sub-folders:

- **`codebert/`:** All the files for the fine-tuned CodeBERT model are stored here.
- **`binary_classifiers/`:** In here there is one sub-folder for each binary classifier, in which the necessary files are stored.

4.1.10 Using binary classifiers vs CodeBERT for detecting KUs

The use of CodeBERT instead of multiple different binary classifiers offers several other benefits apart from the improved accuracy:

(a) It outputs a response that contains binary flags for every KU at once. That way we don't have to get a single output from every binary classifier and then aggregate the results to conclude which KUs were present in a given file.

(b) We only have to load and use 1 model instead of 27^5. This removes the need to work with multiple processes which in turn removes the overhead of starting, stopping and aggregating results from different workers.

(c) CodeBERT is an LLM and has a larger context window than the simpler ML binary classifiers we've trained. Therefore we can work with larger window sizes in the sliding window technique without fear of missing certain KUs. As a result of that, the total number of sliding window iterations over a file is reduced and with that the required time to analyze said file.

4.2 Front-end

The front-end[6] uses React with TypeScript for type safety and code quality. Additionally, Vite is used to provide a fast and efficient development environment, TailwindCSS helps with styling and shadcn/ui provides the UI elements.

As far as packages are concerned, `Axios` facilitates the communication with the back-end API and `ApexCharts` is used to generate the heatmap.

The app allows users to set a GitHub repository URL (other Git repositories, like GitLab, work too) and the number of commits they want to analyze. There are 2 buttons: **"Fetch Commits"** which sends requests to the `/commits` API endpoint and returns a list of commits and **"Extract Skills"** that communicates with the `/analyze` endpoint and presents the results in an interactive heatmap.

[5]This number may vary slightly depending on how many models we end up using.

[6]The source code can be found on GitHub here `https://github.com/NikolasPpd/KU-Detection-Front-End`

In the beginning, before fetching any commits from the back-end, the list of commits on the right side of the screen is empty as seen in Fig. 10

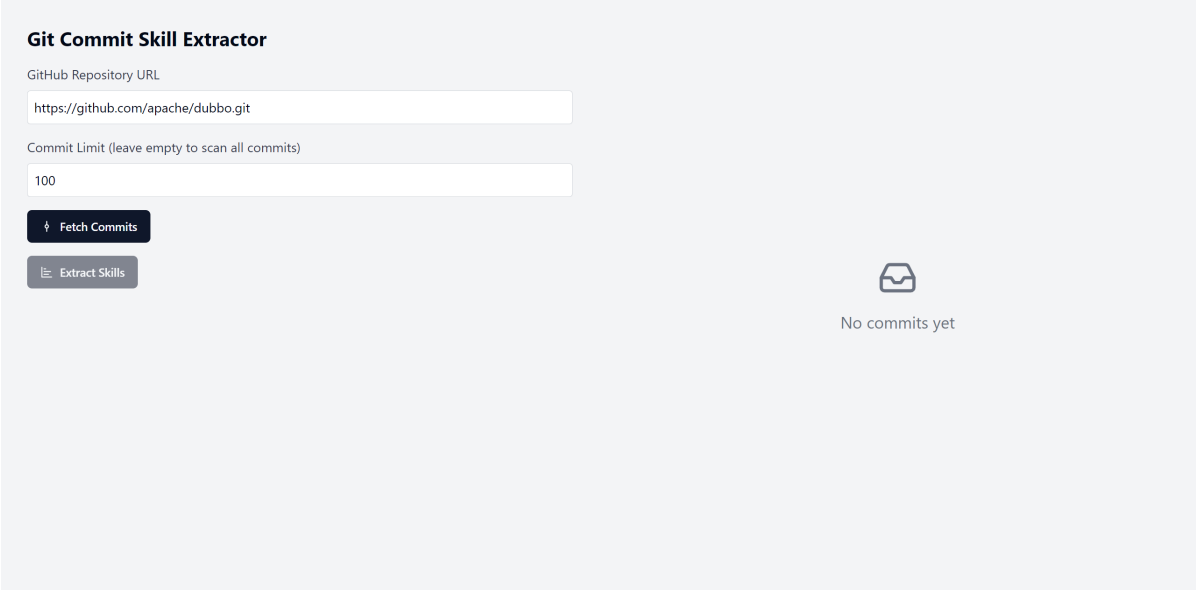

Figure 10: (1) Skill extractor UI before fetching commits from the back-end

Once **"Fetch Commits"** is clicked, a spinner appears inside the button along with a "Please wait" text (Fig. 11). On the right side of the screen, a list of skeleton cards appears until the actual results arrive from the server. Those animated elements provide feedback to the users to let them know that the response is loading.

When the API response is received, the list is populated with card elements (Fig. 12), each of which contains the following information:

- Commit SHA.
- Timestamp of when the commit was created.
- The number of modified Java files in the specified commit.

Also, an empty progress bar appears below the buttons and it displays the total number of modified Java files extracted from the selected commits.

After clicking **"Extract Skills"** the analysis begins and we start receiving responses as described in section 4.1.1. For every response we get the progress bar is incremented accordingly and the heatmap is updated live to incorporate the latest data (Fig. 13).

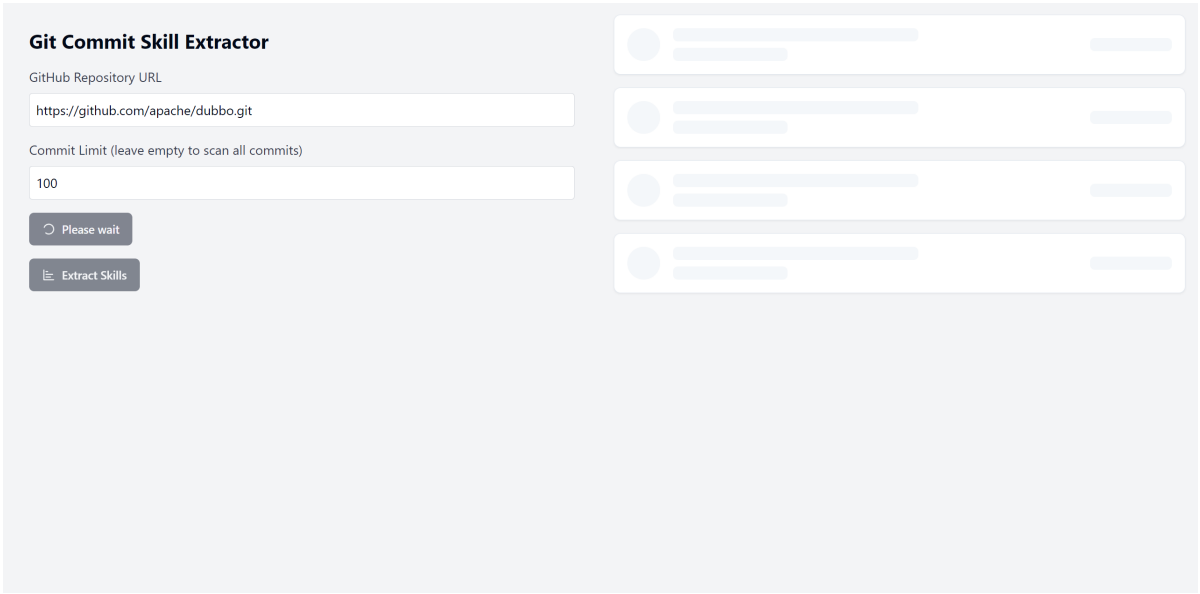

Figure 11: (2) Skill extractor UI after requesting commits from the back-end, but before the results arrive

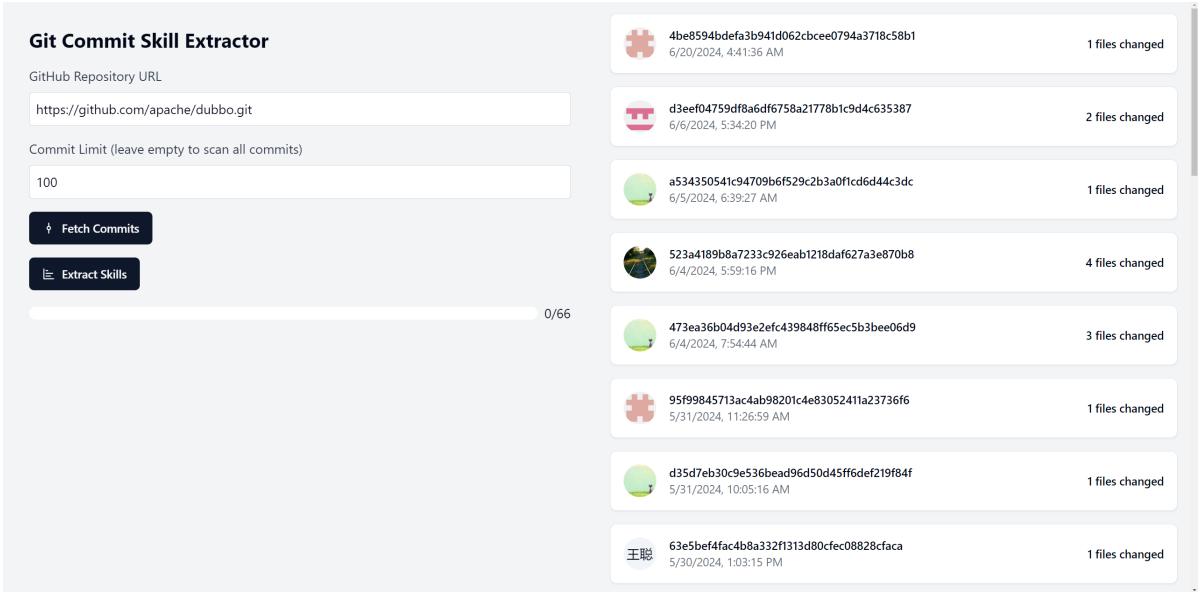

Figure 12: (3) Skill extractor UI after receiving commits from the back-end

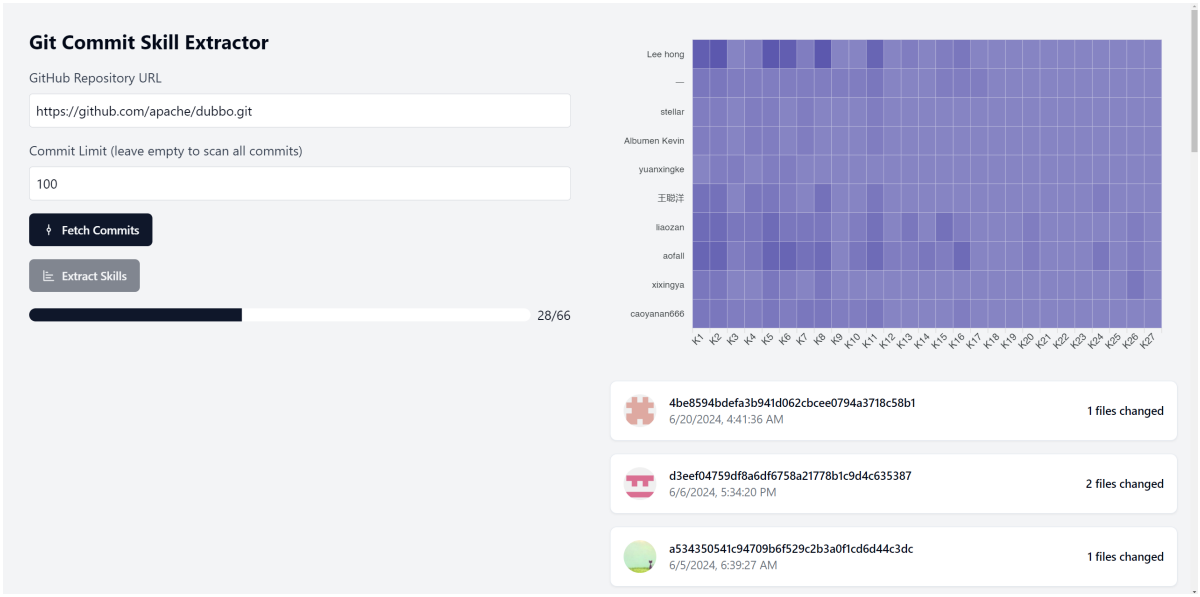

Figure 13: (4) Skill extractor UI during an analysis

When the analysis concludes we have the final version of the heatmap that provides an overview of the skills distribution in the analyzed files (Fig. 14). When hovering over any of the heatmap's tiles a tooltip appears displaying the total number of occurrences of a specific skill for a particular contributor.

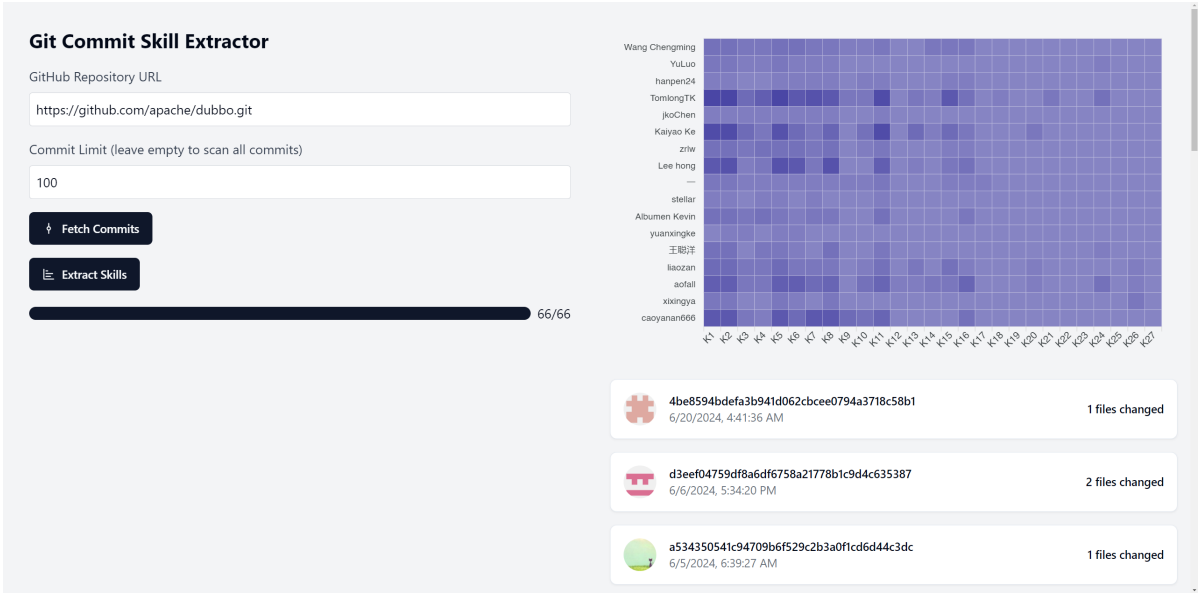

Figure 14: (5) Skill extractor UI after the analysis has been completed

5 Case Study

In this case study, four GitHub repositories were investigated [7]. For each repository, the most recent 450 commits were examined. Our primary objectives were: (1) to assess the knowledge distribution among developers for every KU, (2) to investigate whether there are notable changes in the used KUs over time and (3) to identify potential correlations between various KUs.

From each repository's default active branch, 450 commits were extracted. Merge commits were excluded from the analysis because they do not necessarily reflect the original authorship of the embedded code changes. For the remaining commits, if Java files were present in the diff with added code, the file in its current state was incorporated into the set designated for analysis.

Fig. 15 and Fig. 16 showcase heatmaps illustrating the frequency with which a specific KU was detected in files modified by individual contributors. In instances where an overwhelming majority of changes within the sampled commits were attributed to a single contributor (or a very limited group), the majority of KU detections would predominantly be associated with them. This predominance could overshadow (in terms of presentation) the knowledge distribution profile of other developers. To address this visual imbalance, a predefined threshold of 20 occurrences was set, beyond which it was assumed that the developer had comprehensive understanding of the topic.

Analysis of these heatmaps provides insights into a contributor's activity level and reveals discernible patterns regarding the frequency with which a KU is encountered.

For example, in Fig. 15 and Fig. 16 one can readily observe a small subgroup of developers exhibiting knowledge of almost all Java KUs. The possession of knowledge regarding all KUs, seems to coincide with intense commit activity: Author 2 has performed 524 file changes, while Author 41 has changed 127 files, over the most recent 450 commits in the Apache Dubbo project. On the other hand, dark-colored

[7] https://github.com/dbeaver/dbeaver
https://github.com/apache/dubbo
https://github.com/eclipse/eclipse-collections
https://github.com/jfree/jfreechart

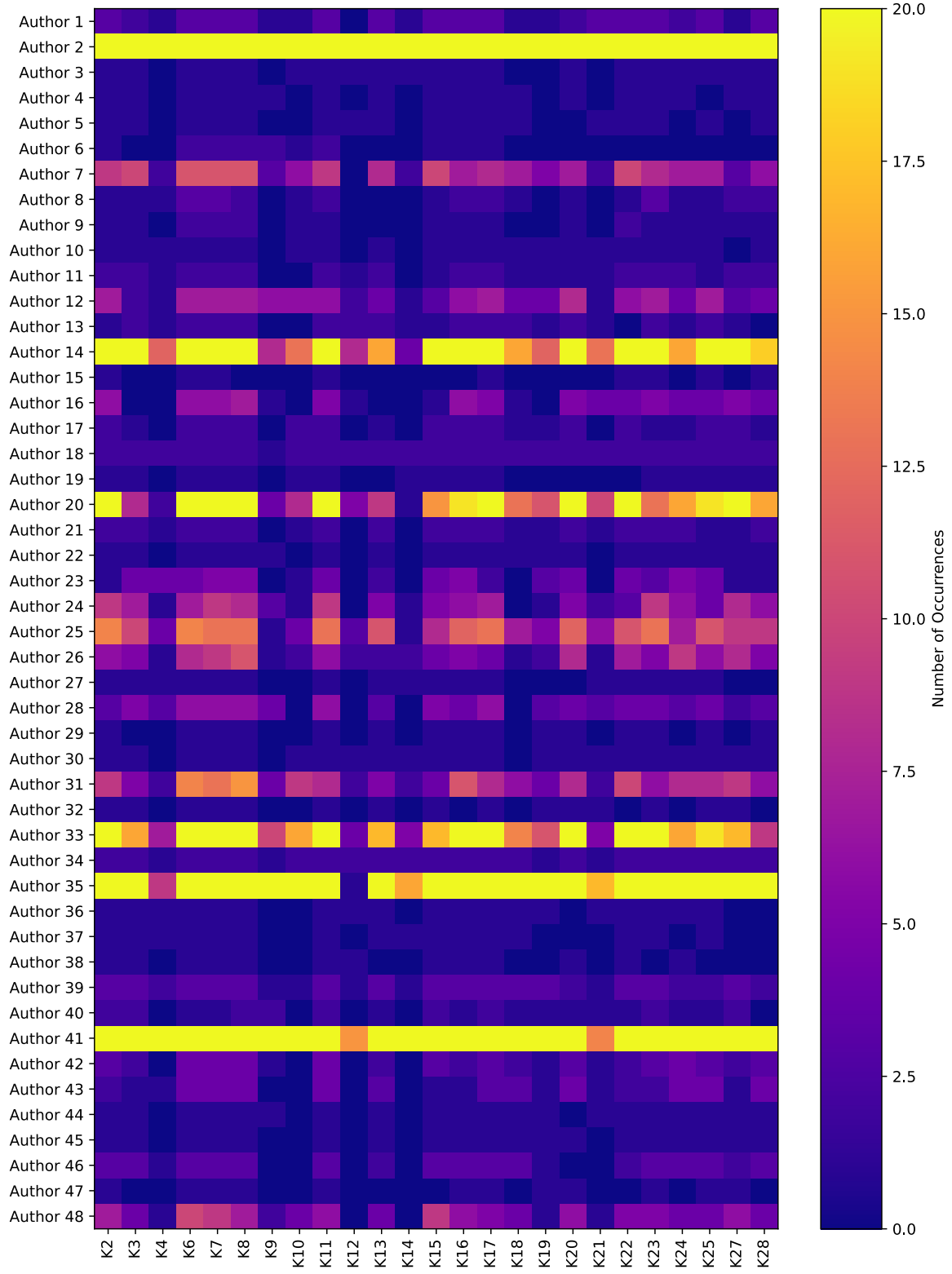

Figure 15: Knowledge distribution among code authors in the Apache Dubbo repository.

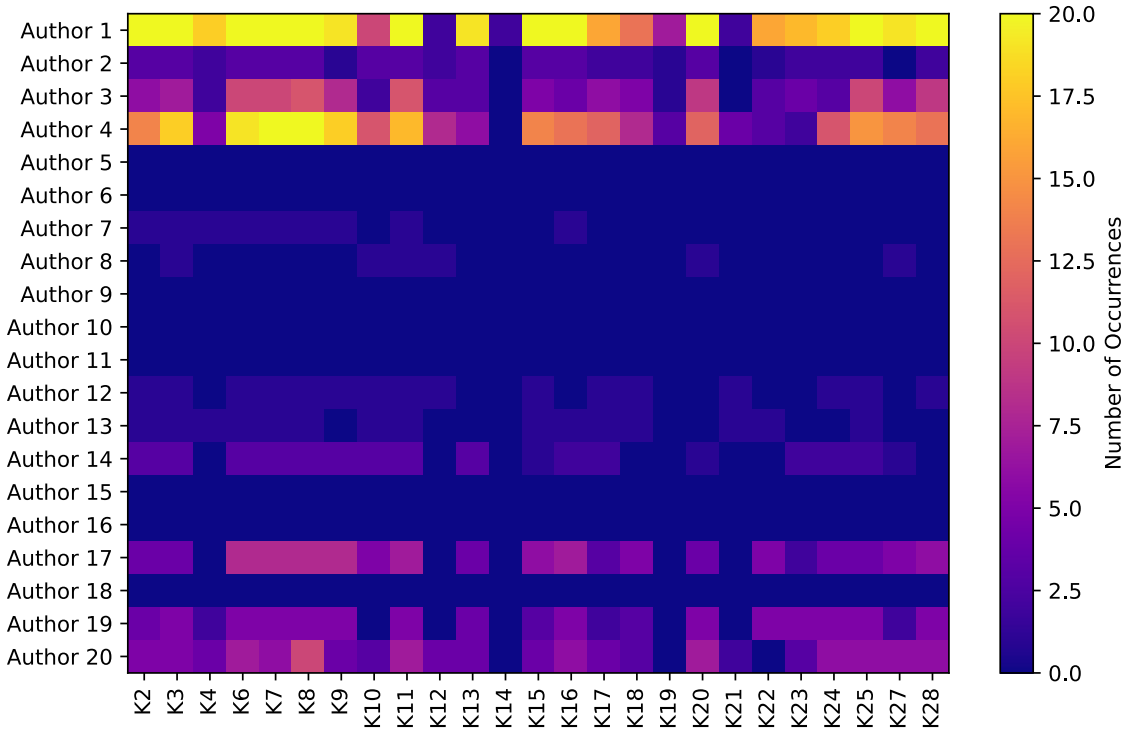

Figure 16: Knowledge distribution among code authors in the Eclipse Collections repository.

vertical "columns" highlight KUs with limited activity. The latter is not necessarily an indicator of skill shortage but may reveal that the corresponding KU is not applicable or useful for that project. For example, that could be the case of K14 (interacting files and directories with the new non-blocking input/output API) and K21 (Java Message Service API) in the Eclipse Collections repository.

The identification of KUs from source code artifacts enables the monitoring of "knowledge evolution" within a project over time. Diving deeper into the Dubbo repository, Fig. 17 visualizes the monthly percentage distribution of various KUs' appearances over a span of 12 months, from October 2022 to September 2023. Each color-coded segment of a bar represents the proportion of a specific KU's appearances relative to the combined appearances of all KUs for a particular month. To derive the data for this chart, we analyzed all commits made to the Dubbo repository over the past year and tallied the occurrences of different KUs. These counts were subsequently grouped by month and converted into percentages, providing a temporal overview of KU usage. By observing the height of these segments across the months, it is possible to discern trends in the usage of individual KUs. Essentially, it provides insights into the rising or waning popularity of each KU over time.

Upon aggregating the findings from all four repositories, Multidimensional Scaling (MDS) was employed to detect any tendencies for KUs to co-occur. Multidimensional scaling is a means of depicting into an abstract Cartesian space the degree of similarity among individual cases of a dataset [25]. The results, illustrated in Fig. 18, are particularly interesting as they highlight three pairs of KUs in close proximity (highlighted in red in the figure):

- Stream API & Java Persistence API (K10/K19): While working with databases, developers seem to retrieve data using JPA and then transform or process that data using the Stream API which provides methods for essential operations like sorting and filtering the data.

- Inheritance & Web Sockets (K6/K25): A plausible rationale for the co-occurrence of these KUs lies in the object-oriented architecture behind WebSockets, which

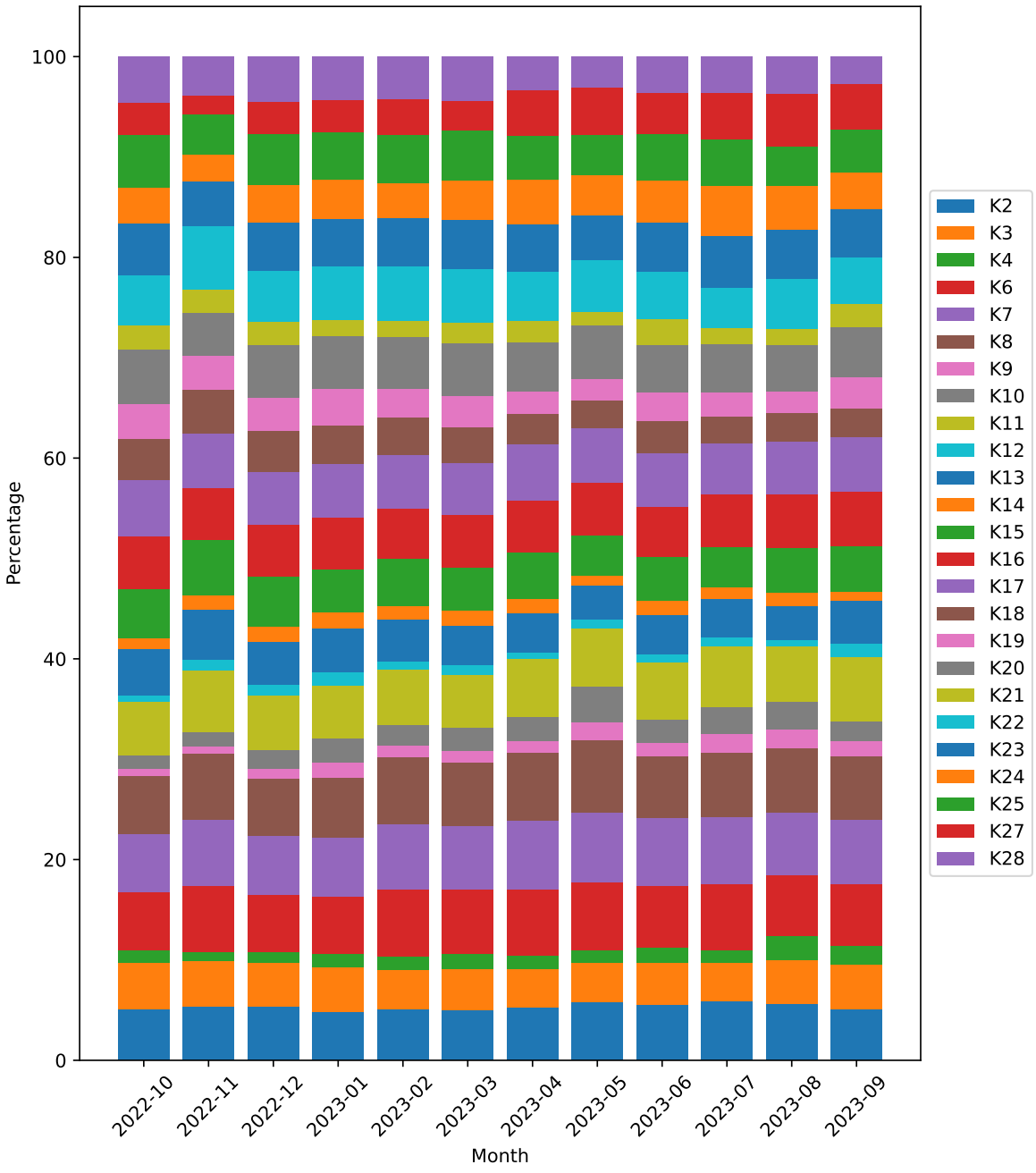

Figure 17: Temporal overview of KU usage in the Dubbo Repository (October 2022 – September 2023)

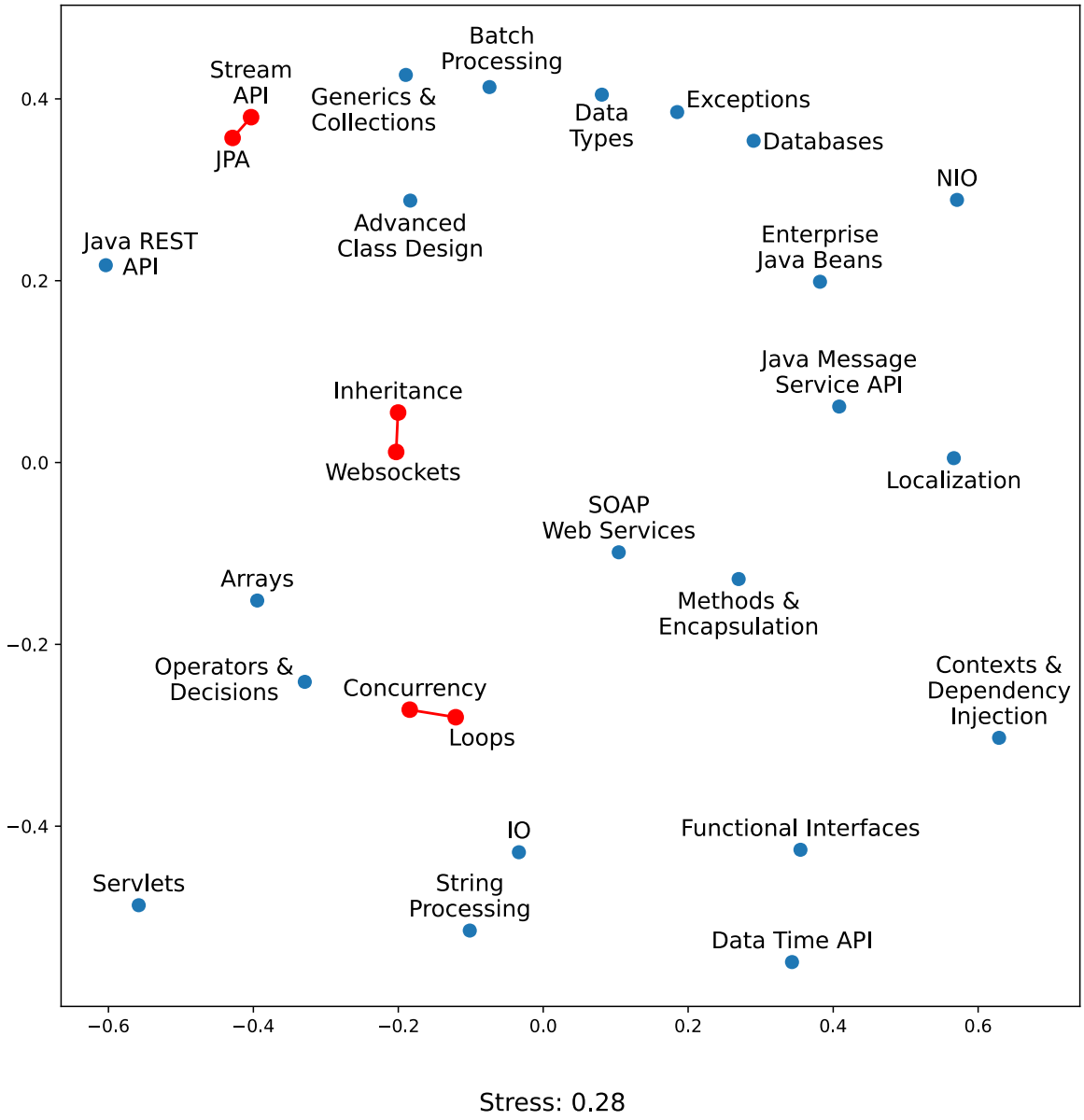

Figure 18: MDS representation of the co-occurrence frequency of KUs highlighting the patterns of their concurrent detection.

often relies on inheritance for creating modular and maintainable code.

- Loops & Concurrency (K4/K16): The parallel Fork/Join Framework often involves splitting of tasks, which can be associated with looping constructs. For example, elements within a collection might be processed concurrently using threads. Loops facilitate data segmentation into smaller units, thread initiation, and synchronization before aggregating and further processing the results.

6 Implications for Practice and Research

Human Resource departments and project managers in software development organizations can benefit from ML-based approaches for the identification of software competencies so as to accurately monitor the supply of skills. Having an accurate understanding of knowledge distribution, not only in terms of high-level skills such as programming languages but also in terms of low-level technological competences such as the knowledge of a particular framework or library, can pinpoint areas in which the supply is short or decreasing. Furthermore, the proximity of competencies, as provided by multidimensional representations, can highlight skills which may be adjacent to a missing one. Considering that classifiers as those proposed in the current study can continuously monitor the possessed skills, hiring can be better planned through informed decision making. Furthermore, directing existing or prospective developers to skills exhibiting a low supply can lead to better reskilling programs. [2]

Through the use of ML-based approaches tailored to artifact-based skill identification, education experts can delve deeper into the knowledge areas where skill shortages are systematically observed. While for some skills the inherent complexity or its rarity might explain such shortage, for other areas it might be related to the gap between the software industry and software engineering education [26]. Software engineering researchers on the other hand, can built on top of such classifiers and improve skill identification by combining them with rule-based detection techniques, especially for rare or novel competencies. Modeling the knowledge space within an organization in a form of networks can reveal clusters of developers cooperating within the context of specific competencies or even developers acting as "bridges" among different technologies. Sophisticated user interfaces can convey information related to existing and missing competencies to assist HR departments in recruitment and training programs as well as performance management.

7 Limitations and Threats to Validity

In this section we discuss limitations pertaining to the proposed approach for extracting skills from source code artifacts and threats to the validity of the case study findings [27].

While the proposed approach targets the identification of Knowledge Units [2] within Java source code files, the use of binary classifiers trained via a one-vs-rest strategy can be extended to other technological skills as well as other programming languages. We acknowledge that the assumption that a developer committing changes to a particular code file possesses knowledge of the entire file's content is a rather strong one which does not necessarily hold, especially for large files or files that underwent a number of changes in their history. To address this limitation we plan to integrate slicing techniques, so as to extract a backward (or forward) slice for the variables affected by a given change, so as to increase the programming context, which is then fed to a classifier.

As in any classification task, training presupposes of a labeled dataset where each sample is tagged with all its pertinent KUs. In the proposed approach we leveraged ChatGPT by prompting it so as to generate code snippets. While the potential of LLMs to generate examples relevant to a prompt is very high, screening of the returned snippets to ensure a high quality training dataset is required. This approach poses limitations but can be complemented by manually tagged code snippets, especially within a large organization where the "wisdom of the crowd" can be sought.

The KU detection algorithm has been evaluated using a subset of 22,981 code files from 22 GitHub repositories. While the reported performance measures are only meant to serve as an indicator of the potential of the approach, we cannot argue that the results can be generalized to a wider population of Java projects. Different applications, domains and programming conventions might render the identification of KUs more challenging. Similar external validity threats apply to the results reported for the knowledge distribution and the co-occurrence frequency of KUs. Since the proposed approach would be meaningful in the context of a particular company or

open-source organization, we believe that these external validity threats can be safely mitigated by training the classifiers on a representative dataset of interest.

In terms of construct validity, threats arise from the use of KUs for representing skills in the Java programming language. As depicted in Table 7 KUs may refer to fundamental programming language constructs which can spread over an entire piece of code or to the use of more specific classes, methods and blocks. As a consequence discerning some KUs is more challenging while others might not even be regarded as "skills" by most programmers. Nevertheless, we opted for using a pre-defined set of key capabilities stemming from Oracle's Java certification exam topics rather than defining an arbitrary set of Java programming skills. In any case, the approach can be tailored to the needs of an organization by retaining in the classification models only the key capabilities of interest.

Assuming that a commit involving changes to a file implies knowledge of the entire file, poses threats to internal validity: Generalizing from the changed/added lines of code to the entire file context introduces unanticipated factors (programming constructs) affecting the classification result. We consider this to be both a limitation as well as a threat to internal validity affecting the accuracy of knowledge distribution among code authors. As mentioned above, slicing techniques can narrow down the scope of the code fragment that is fed as input to the classifier.

Finally, to mitigate reliability validity threats which are related to the ease by which the current study could be replicated, we: a) outline all steps of the proposed approach and b) provide a replication package online[8].

[8] https://zenodo.org/records/10056038

8 Future Improvements

8.1 Dataset

In the context of this thesis, manually checking and labeling all 90,144[9] code samples in the dataset and the other 22,981 extracted from GitHub repositories was infeasible. Thus, we resorted to using regular expressions to look for specific keywords and structures in the code and then decide if a certain KU is present or not. While this provided a decent approximation to the actual ground truth of the dataset, ideally we'd want to have all samples manually checked by humans.

8.2 Model Training

Further experimentation with various different NNs would be useful to see if models that are simpler and lighter than CodeBERT can perform equally as well.

For CodeBERT, training it for more epochs along with early stopping could allow us to unlock better accuracy and overall performance.

8.3 Code Slicing

To deal with the assumption that a contributor that made even the slightest modification in a code file possesses complete knowledge of that file, we would like to employ code slicing. This would allow us to narrow down the examined portion of the file and stay closer to the parts most relevant to the modified lines. By focusing only on the most relevant code changes we can avoid attributing KUs to contributors that do not possess that knowledge.

8.4 Back-end

For the back-end the most important addition would be to integrate a database that would persistently store analysis results for future use. Currently, to keep things simple, all data saved on the server is kept in-memory. If the server crashes or is terminated everything is deleted and previously analyzed commits need to be analyzed

[9] 71,135 after removing duplicates

again. By caching analyses' results we can drastically decrease execution time and use of computational resources.

Another functionality that can be implemented is real-time repository monitoring. It is a service running constantly and periodically checking the selected repositories for new commits. Once a new commit has been found it can be retrieved, have its files analyzed and their results added to the database. These changes would then be reflected live on any clients using the tool during that moment.

8.5 Front-end

One crucial improvement for the front-end is to support a more flexible way of viewing all the available commits and selecting which ones we want to analyze. Allowing users to visually select commits would provide a better user experience. Going one step further, we could also add filters to allow for example to select all commits within a specific date range or from the same contributor.

Adding more types of charts and graphs would make it easier to showcase all the underlying knowledge that exists within the system. Different charts could provide different insights; for example a temporal evolution chart would be able to showcase the usage of specific skills over time.

Lastly, it is important to add data export functionality to allow users to export specific data to CSV or JSON files for further analysis.

9 Conclusion

Assessing the skills possessed by software professionals and contrasting them to the needs of a software development organization is of paramount importance for identifying or predicting skill shortages. The dynamic landscape of emerging software technologies renders the conventional approaches for assessing skills inadequate, as interviews and programming tests capture a specific time point in someone's evolving career in IT. To address this shortcoming, in this thesis we systematically investigate the potential of Machine Learning and Neural Networks in identifying competences from source code artifacts.

More specifically, we analyzed the performance of binary ML classifiers in identifying Knowledge Units in Java code files, after having trained the corresponding models on 90,144 code snippets generated with the help of ChatGPT (GPT-3.5). The obtained accuracy of the models opens up the possibility of getting insight into the distribution of knowledge among contributors in a software project, the evolution of skills over time and the proximity of competencies. Such analyses can assist software development teams and organizations in profiling their personnel in terms of skills, recognize skill shortages, recommend reskilling opportunities and perform better hiring of new developers.

Aditionally, we experimented with the use of CodeBERT, an LLM pre-trained on Java (among other programming languages). It proved to be the ideal choice for our use case; phenomenal performance and speed compared to the binary classifiers while keeping hardware requirement low, in contrast to other LLMs that are popular today.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," 2023.

[2] M. Ahasanuzzaman, G. A. Oliva, and A. E. Hassan, "Using knowledge units of programming languages to recommend reviewers for pull requests: An empirical study," *CoRR*, vol. abs/2305.05654, 2023.

[3] "Employment and social developments in europe: Addressing labour shortages and skills gaps in the eu," 2023.

[4] D. Akdur, "Analysis of software engineering skills gap in the industry," *ACM Transactions on Computing Education*, vol. 23, pp. 1–28, Dec. 2022.

[5] "Nash squared digital leadership report 2022," 2022.

[6] M. Flauzino, M. Souza, V. Durelli, and R. Durelli, "What do software team managers want from a skills identification?," in *Anais do IX Workshop de Visualização, Evolução e Manutenção de Software*, (Porto Alegre, RS, Brasil), pp. 16–20, SBC, 2021.

[7] N. Bibi, Z. Anwar, and T. Rana, "Expertise based skills management system to support resource allocation," *PLOS ONE*, vol. 16, pp. 1–20, 08 2021.

[8] M. . Company, "Beyond hiring: How companies are reskilling to address talent gaps," 2 2020.

[9] G. J. Greene and B. Fischer, "Cvexplorer: identifying candidate developers by mining and exploring their open source contributions," in *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering*, ASE '16, (New York, NY, USA), p. 804{809, Association for Computing Machinery, 2016.

[10] C. Teyton, M. Palyart, J.-R. Falleri, F. Morandat, and X. Blanc, "Automatic extraction of developer expertise," in *Proceedings of the 18th International Conference*

on Evaluation and Assessment in Software Engineering, EASE '14, (New York, NY, USA), Association for Computing Machinery, 2014.

[11] E. Constantinou and G. M. Kapitsaki, "Identifying developers' expertise in social coding platforms," in *2016 42th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, pp. 63–67, 2016.

[12] K. C. Nguyen, M. Zhang, S. Montariol, and A. Bosselut, "Rethinking skill extraction in the job market domain using large language models," 2024.

[13] D. Nam, A. Macvean, V. Hellendoorn, B. Vasilescu, and B. Myers, "Using an llm to help with code understanding," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ICSE '24, (New York, NY, USA), Association for Computing Machinery, 2024.

[14] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating large language models trained on code," 2021.

[15] S. Sahar, M. Younas, M. M. Khan, and M. U. Sarwar, "Dp-ccl: A supervised contrastive learning approach using codebert model in software defect prediction," *IEEE Access*, vol. 12, pp. 22582–22594, 2024.

[16] R. L. Russell, L. Kim, L. H. Hamilton, T. Lazovich, J. A. Harer, O. Ozdemir, P. M. Ellingwood, and M. W. McConley, "Automated vulnerability detection in source code using deep representation learning," 2018.

[17] J. Li, P. He, J. Zhu, and M. R. Lyu, "Software defect prediction via convolutional neural network," in *2017 IEEE International Conference on Software Quality, Reliability and Security (QRS)*, pp. 318–328, 2017.

[18] P. T. Nguyen, J. Di Rocco, C. Di Sipio, R. Rubei, D. Di Ruscio, and M. Di Penta, "Gptsniffer: A codebert-based classifier to detect source code written by chatgpt," *Journal of Systems and Software*, vol. 214, p. 112059, 2024.

[19] S. Kourtzanidis, A. Chatzigeorgiou, and A. Ampatzoglou, "Reposkillminer: Identifying software expertise from github repositories using natural language processing," in *35th IEEE/ACM International Conference on Automated Software Engineering, ASE 2020, Melbourne, Australia, September 21-25, 2020*, pp. 1353–1357, IEEE, 2020.

[20] D. Atzberger, T. Cech, A. Jobst, W. Scheibel, D. Limberger, M. Trapp, and J. Döllner, "Visualization of knowledge distribution across development teams using 2.5d semantic software maps," in *17th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications, VISIGRAPP 2022, February 6-8, 2022*, pp. 210–217, SCITEPRESS, 2022.

[21] X. Ye, Y. Zheng, W. Aljedaani, and M. W. Mkaouer, "Recommending pull request reviewers based on code changes," *Soft Comput.*, vol. 25, no. 7, pp. 5619–5632, 2021.

[22] O. C. da Costa Castro, G. Avelino, P. de A. Santos Neto, R. Britto, and M. T. Valente, "Identifying source code file experts," in *ESEM '22: ACM / IEEE International Symposium on Empirical Software Engineering and Measurement, Helsinki Finland, September 19 - 23, 2022*, pp. 125–136, ACM, 2022.

[23] J. E. Montandon, L. L. Silva, and M. T. Valente, "Identifying experts in software libraries and frameworks among github users," in *Proceedings of the 16th International Conference on Mining Software Repositories, MSR 2019, 26-27 May 2019, Montreal, Canada*, pp. 276–287, IEEE / ACM, 2019.

[24] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "Codebert: A pre-trained model for programming and natural languages," 2020.

[25] W. S. Torgerson, "Multidimensional scaling: I. theory and method," *Psychometrika*, vol. 17, pp. 401–419, Dec. 1952.

[26] D. Oguz and K. Oguz, "Perspectives on the gap between the software industry and the software engineering education," *IEEE Access*, vol. 7, pp. 117527–117543, 2019.

[27] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Springer US, 2000.

A Appendix

Table 7: Knowledge Units (KUs) and their corresponding Key Capabilities (KCs) [2]

Knowledge Unit (KU)	**Key Capabilities (KCs)**
[K1] Data Types	**[C1]** Declare and initialize different types of variables (e.g., primitive, parameterized and array type), including the casting of primitive data types
[K2] Operators and Decisions	**[C1]** Use Java operators (e.g., assignment and postfix operators); use parentheses to override operator precedence **[C2]** Test equality between strings and other objects using **==** and **`equals()`** **[C3]** Create and use **`if`**, **`if-else`**, and ternary constructs **[C4]** Use the **`switch`** statement
[K3] Arrays	**[C1]** Declare, instantiate, initialize and use a one-dimensional array **[C2]** Declare, instantiate, initialize and use a multi-dimensional array
[K4] Loops	**[C1]** Create and use **`while`** loops **[C2]** Create and use **`for`** loops, including the **`enhanced for`** loop **[C3]** Create and use **`do-while`** loops **[C4]** Use the **`break`** statement **[C5]** Use the **`continue`** statement
[K5] Methods and Encapsulation	**[C1]** Create methods with arguments and return values **[C2]** Apply the **`static`** keyword to methods, fields, and blocks **[C3]** Create an overloaded method and overloaded constructor **[C4]** Create a constructor chaining (use **`this()`** method to call one constructor from another constructor) **[C5]** Use variable length arguments in methods **[C6]** Use different access modifiers (e.g., **`private`** and **`protected`**) other than **`default`** **[C7]** Apply encapsulation: identify set and get methods to initialize any private class variables **[C8]** Apply encapsulation: Immutable class generation-final class and initialize private variables through the constructor

Knowledge Unit (KU)	Key Capabilities (KCs)
[K6] Inheritance	**[C1]** Use basic polymorphism (e.g., a superclass refers to a subclass) **[C2]** Use polymorphic parameters (e.g., pass instances of a subclass or interface to a method) **[C3]** Create overridden methods **[C4]** Create **`abstract`** classes and **`abstract`** methods **[C5]** Use **`super()`** and the **`super`** keyword to access the members (e.g., fields and methods) of a parent class **[C6]** Use casting in referring a subclass object to a superclass object
[K7] Advanced Class Design	**[C1]** Create inner classes, including static inner classes, local classes, nested classes, and anonymous inner classes **[C2]** Develop code that uses the **`final`** keyword **[C3]** Use enumerated types including methods and constructors in an **`enum`** type **[C4]** Create singleton classes and immutable classes
[K8] Generics and Collections	**[C1]** Create and use a generic class **[C2]** Create and use **`ArrayList`**, **`TreeSet`**, **`TreeMap`**, and **`ArrayDeque`** **[C3]** Use **`java.util.Comparator`** and **`java.lang.Comparable`** interfaces **[C4]** Iterate using the **`forEach`** method of **`List`**
[K9] Functional Interfaces	**[C1]** Use the built-in interfaces included in the **`java.util.function`** packages such as **`Predicate`**, **`Consumer`**, **`Function`**, and **`Supplier`** **[C2]** Develop code that uses primitive versions of functional interfaces **[C3]** Develop code that uses binary versions of functional interfaces **[C4]** Develop code that uses the **`UnaryOperator`** interface
[K10] Stream API	**[C1]** Develop code to extract data from an object using the **`peek()`** and **`map()`** methods, including primitive versions of the **`map()`** method **[C2]** Search for data by using search methods of the **`Stream`** classes, including **`findFirst`**, **`findAny`**, **`anyMatch`**, **`allMatch`**, **`noneMatch`** **[C3]** Develop code that uses the **`Optional`** class **[C4]** Develop code that uses **`Stream`** data methods and calculation methods **[C5]** Sort a collection using the Stream API **[C6]** Save results to a collection using the **`collect()`** method **[C7]** Use the **`flatMap()`** method in the Stream API

Knowledge Unit (KU)	Key Capabilities (KCs)
[K11] Exceptions	**[C1]** Create a **`try-catch`** block **[C2]** Use the **`catch`**, **`multi-catch`**, and **`finally`** clauses **[C3]** Auto-close resources with a **`try-with-resources`** statement **[C4]** Create custom exceptions and autocloseable resources **[C5]** Create and invoke a method that throws an exception **[C6]** Use common exception classes and categories (such as **`NullPointerException`**, **`ArithmeticException`**, **`ArrayIndexOutOfBoundsException`**, **`ClassCastException`**) **[C7]** Use assertions
[K12] Date Time API	**[C1]** Create and manage date-based and time-based events including a combination of date and time into a single object using **`LocalDate`**, **`LocalTime`**, **`LocalDateTime`**, **`Instant`**, **`Period`**, and **`Duration`** **[C2]** Format date and time values when using different timezones **[C3]** Create and manage date-based and time-based events using **`Instant`**, **`Period`**, **`Duration`**, and **`Temporal Unit`** **[C4]** Create and manipulate calendar data using classes from **`java.time.LocalDateTime`**, **`java.time.LocalDate`**, **`java.time.LocalTime`**, **`java.time.format.DateTimeFormatter`**, and **`java.time.Period`**
[K13] IO	**[C1]** Read and write data using the console **[C2]** Use **`BufferedReader`**, **`BufferedWriter`**, **`File`**, **`FileReader`**, **`FileWriter`**, **`FileInputStream`**, **`FileOutputStream`**, **`ObjectOutputStream`**, **`ObjectInputStream`**, and **`PrintWriter`** from the **`java.io`** package
[K14] NIO	**[C1]** Use the **`Path`** interface to operate on file and directory paths **[C2]** Use the **`Files`** class to check, read, delete, copy, move, and manage metadata of a file or directory
[K15] String Processing	**[C1]** Search, parse and build strings **[C2]** Manipulate data using the **`StringBuilder`** class and its methods **[C3]** Use regular expressions using the **`Pattern`** and **`Matcher`** classes **[C4]** Use string formatting

Knowledge Unit (KU)	Key Capabilities (KCs)
[K16] Concurrency	**[C1]** Create worker threads using **`Runnable`**, **`Callable`** and use an **`ExecutorService`** to concurrently execute tasks **[C2]** Use the **`synchronized`** keyword and the **`java.util.concurrent.atomic`** package to control the order of thread execution **[C3]** Use **`java.util.concurrent`** collections and classes including **`CyclicBarrier`** and **`CopyOnWriteArrayList`** **[C4]** Use the parallel Fork/Join Framework
[K17] Databases	**[C1]** Describe the interfaces that make up the core of the JDBC API, including the **`Driver`**, **`Connection`**, **`Statement`**, and **`ResultSet`** interfaces **[C2]** Submit queries and read results from the database, including creating statements, returning result sets, iterating through the results, and properly closing result sets, statements, and connections
[K18] Localization	**[C1]** Read and set the locale by using the **`Locale`** object **[C2]** Build a resource bundle for each locale and load a resource bundle in an application
[K19] Java Persistence API	**[C1]** Create JPA Entities and Object-Relational Mappings (ORM) **[C2]** Use **`EntityManager`** to perform database operations, transactions, and locking with JPA entities **[C3]** Create and execute JPQL statements
[K20] Enterprise Java Beans	**[C1]** Create session EJB components containing synchronous and asynchronous business methods, manage the life cycle container callbacks, and use interceptors **[C2]** Create EJB timers
[K21] Java Message Service API	**[C1]** Implement Java EE message producers and consumers, including message-driven beans **[C2]** Use transactions with the JMS API
[K22] SOAP Web Services	**[C1]** Create SOAP Web Services and Clients using the JAX-WS API **[C2]** Create, marshall and unmarshall Java Objects by using the JAXB API
[K23] Servlets	**[C1]** Create Java Servlets and use HTTP methods **[C2]** Handle HTTP headers, parameters and cookies **[C3]** Manage servlet life cycle with container callback methods and WebFilters
[K24] Java REST API	**[C1]** Apply REST service conventions **[C2]** Create REST Services and clients using the JAX-RS API

Knowledge Unit (KU)	Key Capabilities (KCs)
[K25] Websockets	**[C1]** Create WebSocket Server and Client Endpoint Handlers **[C2]** Produce and consume, encode and decode WebSocket messages
[K26] Java Server Faces	**[C1]** Use JSF syntax and JSF Tag Libraries **[C2]** Handle localization and produce messages **[C3]** Use Expression Language (EL) and interact with CDI beans
[K27] Contexts and Dependency Injection	**[C1]** Create CDI Bean Qualifiers, Producers, Disposers, Interceptors, Events, and Stereotypes
[K28] Batch Processing	**[C1]** Implement batch jobs using the JSR 352 API